

Vysoká škola ekonomická v Praze
Fakulta informatiky a statistiky



Návrh a testování prostředí pro AI agenty v softwarovém vývoji

BAKALÁŘSKÁ PRÁCE

Studijní program: Aplikovaná informatika

Autor: Thanh An Nguyen

Vedoucí práce: Ing. Jiří Korčák

Praha, květen 2026

Prohlášení

Prohlašuji, že jsem bakalářskou práci zpracoval samostatně a že jsem uvedl všechny použité prameny a literaturu, ze kterých jsem čerpal.

V Praze dne 24. března 2026

Thanh An Nguyen

Poděkování

Poděkování.

Abstrakt

AI coding agenti dokáží generovat funkční kód, ale bez vhodných instrukcí nedodržují softwarově inženýrské praktiky. Existující benchmarky měří pouze výsledek (vyřešil agent úkol?), nikoli kvalitu procesu ani kódu. Formalizovaný postup pro návrh a vyhodnocování instrukcí dosud chybí. Práce si klade tři cíle: (1) navrhnout sadu metrik měřících proces, kvalitu a efektivitu práce agenta; (2) demonstrovat iterativní postup návrhu instrukcí řízený těmito metrikami; (3) ablacemi identifikovat, které složky instrukční sady jsou nezbytné. Metodika kombinuje návrhový výzkum (DSR) s případovou studií systému upomínek faktur. Agent (model Minimax prostřednictvím nástroje OpenCode) prošel pěti pilotními iteracemi a čtyřmi ablačními běhy. Chování bylo měřeno 19 metrikami ve třech kategoriích: procesní (P), produktové (Q) a efektivita (E). Metriky zachytily zlepšení ze 4/10 na 9/10 splněných kritérií během tří iterací. Analýza odhalila vzorec vývoje instrukcí: obecné pravidlo → konkrétní příkaz → verifikační krok. Ablace ukázaly, že verifikační kroky nejsou redundantní — jejich odebrání zhoršilo čtyři metriky. Sekce kódových konvencí byla pro automatizované metriky převážně redundantní. Nedeterminismus modelu se ukázal jako dominantní faktor variability procesní compliance. Přenositelným přínosem jsou sada metrik a iterativní postup; konkrétní naměřené hodnoty jsou vázány na podmínky případové studie.

Klíčová slova

AI coding agent, instrukční sada, metriky kvality software, iterativní návrh, ablační studie

Abstract

AI coding agents can generate functional code, but without proper instructions they fail to follow software engineering practices. Existing benchmarks measure only the outcome (did the agent solve the task?), not process quality or code quality. A formalized process for designing and evaluating instructions is still lacking. This thesis pursues three goals: (1) design a metric suite measuring agent process, quality, and efficiency; (2) demonstrate an iterative instruction design process driven by these metrics; (3) use ablation to identify which instruction components are necessary. The methodology combines Design Science Research (DSR) with a single case study of a dunning system. The agent (Minimax model via the OpenCode tool) underwent five pilot iterations and four ablation runs. Behavior was measured using 19 metrics in three categories: process (P), product (Q), and efficiency (E). The metrics captured improvement from 4/10 to 9/10 satisfied criteria over three iterations. Analysis revealed an instruction evolution pattern: general rule → specific command → verification

step. Ablation showed that verification steps are not redundant—removing them degraded four metrics. The code convention section proved largely redundant for automated metrics. Model non-determinism emerged as the dominant factor in process compliance variability. The transferable contributions are the metric suite and the iterative process; the specific measured values are bound to the case study conditions.

Keywords

AI coding agent, instruction set, software quality metrics, iterative design, ablation study

Obsah

Úvod	11
1 Vymezení problému a cílů práce	12
1.1 Motivace	12
1.2 Cíle práce	12
1.3 Rozsah práce	13
2 Teoretická východiska	14
2.1 Softwarové inženýrství a životní cyklus	15
2.1.1 Definice a komplexita software	15
2.1.2 Fáze životního cyklu	15
2.1.3 Modely a metodiky	15
2.1.4 Měření kvality software	15
2.2 AI coding agenti a scaffolding	15
2.2.1 Základní pojmy	15
2.2.2 Typy coding agentů	15
2.2.3 Jak agenti mění SDLC	15
2.2.4 Problém kontextu	15
2.2.5 Přístupy k memory a kontextu	15
2.2.6 Struktura instrukcí	15
2.3 Evaluace agentů a instrukcí	15
2.3.1 Hodnocení agentů a benchmarky	15
2.3.2 Evaluace instrukcí	15
3 Metodika	16
3.1 Návrhový výzkum (DSR)	16
3.2 Sada metrik	17
3.2.1 Procesní metriky (P1–P8)	17
3.2.2 Produktové metriky (Q1–Q8)	18
3.2.3 Metriky efektivity (E1–E3)	20
3.2.4 Aplikace LLM-as-judge	20
3.3 Experimentální design	21
3.3.1 Výběr projektu	21
3.3.2 Fixní proměnné	21
3.3.3 Iterativní cyklus	22
3.3.4 Pilotní fáze	23
3.3.5 Komparativní variace	23
3.4 Přehledová tabulka	24
3.5 Omezení a validita	25

4 Praktická část	27
4.1 Příprava experimentu	27
4.1.1 Konstrukce specifikace	27
4.1.2 Referenční implementace	28
4.1.3 Konstrukce baseline instrukcí	29
4.2 Pilotní iterace	31
4.2.1 Pilot-r1: baseline	31
4.2.2 Pilot-r2	33
4.2.3 Pilot-r3	35
4.2.4 Pilot-r4	36
4.2.5 Pilot-r5: verifikace kontraktu	38
4.3 Komparativní variace	41
4.3.1 Ablace A: bez verifikačních kroků	43
4.3.2 Ablace B: bez Package Quality	44
4.3.3 Závěr komparativní fáze	45
4.4 Souhrnné výsledky	46
5 Vyhodnocení a diskuse	48
5.1 Interpretace výsledků	48
5.1.1 Cíl 1: Sada metrik	48
5.1.2 Cíl 2: Iterativní postup	49
5.1.3 Cíl 3: Ablace	50
5.2 Porovnání s literaturou	50
5.3 Limity ve světle výsledků	52
5.4 Doporučení pro praxi	53
5.5 Náměty pro další výzkum	54
Závěr	56
Bibliografie	57
A Formulář v plném znění	61
B Zdrojové kódy výpočetních procedur	62
C Baseline AGENTS.md	63

Seznam obrázků

4.1	Baseline AGENTS.md (pilot-r1) — kompletní znění instrukční sady použité jako výchozí bod pilotních iterací.	30
4.2	Vizuální diff změn AGENTS.md mezi pilot-r1 a pilot-r2	33
4.3	Vizuální diff změn AGENTS.md mezi pilot-r2 a pilot-r3	34
4.4	Vizuální diff změn AGENTS.md mezi pilot-r3 a pilot-r4	36
4.5	Vizuální diff změn AGENTS.md mezi pilot-r3 a pilot-r5	38
4.6	Souhrnná evoluce AGENTS.md : pilot-r1 → pilot-r3. Zelené řádky = přidáno, červené = odebráno.	40
4.7	Změny AGENTS.md pro ablaci A: odebrány verifikační kroky (git log , tsc , eslint) z Process	42
4.8	Změny AGENTS.md pro ablaci B: odebrána celá sekce Package Quality (konvence, strict TypeScript, JSDoc, čisté API).	42

Seznam tabulek

3.1	Sada metrik: kód, měřená vlastnost, nástroj, exit kritérium a typ	24
4.1	Mapování sekcí baseline AGENTS.md na metriky	31
4.2	Pilot-r1: naměřené metriky	32
4.3	Pilot-r2: metriky s změnou oproti r1	33
4.4	Pilot-r3: metriky s změnou oproti r2	35
4.5	Pilot-r4: metriky s změnou oproti r3	36
4.6	Pilot-r5: metriky s změnou oproti r3	38
4.7	Ablace A: srovnání s r3 (bez verifikačních kroků)	43
4.8	Ablace B: srovnání s r3 (bez sekce Package Quality)	44
4.9	Souhrnné výsledky všech běhů (pilot + ablance)	46

Seznam použitých zkratek

TODO Doplňte zkratky a tento řádek smažte.

Úvod

[DRAFT]

AI agenti se stávají součástí softwarového vývoje: dokáží implementovat funkcionalitu, psát testy, spravovat verzovací historii a komunikovat prostřednictvím issues a pull requestů. Kvalita jejich výstupu však závisí na instrukcích, které dostanou — na instrukční sadě definující pracovní postup, omezení a kontext projektu. Jak tyto instrukce systematicky navrhovat a jak měřit, zda agent dodržuje požadované praktiky, zůstává otevřenou otázkou.

Tato práce navrhuje sadu 19 metrik ve třech kategoriích — procesní (P1–P8), produktové (Q1–Q8) a metriky efektivity (E1–E3) — které měří nejen výsledek, ale i proces a kvalitu práce agenta. Na případové studii systému upomínek faktur demonstruje iterativní postup návrhu instrukcí s využitím těchto metrik a komparativními variacemi (ablacemi) identifikuje, které složky instrukční sady přispívají k měřenému chování agenta. Metodicky práce vychází z návrhového výzkumu (Design Science Research) s evaluací formou případové studie jednoho projektu (Yin, 2018).

Práce je členěna do pěti kapitol. Kapitola 1 vymezuje problém, formuluje tři cíle práce a ohraňuje její rozsah. Kapitola 2 shrnuje teoretická východiska: softwarové inženýrství a životní cyklus vývoje, AI coding agenty, scaffolding a instrukce a měření kvality softwaru. Kapitola 3 popisuje metodiku — návrhový výzkum, sadu metrik, experimentální design a omezení validity. Kapitola 4 zachycuje průběh případové studie: pilotní fázi pěti iterací a komparativní variace. Kapitola 5 interpretuje výsledky ve vztahu k cílům práce, porovnává je s existujícím výzkumem a diskutuje limity.

1. Vymezení problému a cílů práce

1.1 Motivace

[DRAFT]

AI agenti se používají k vývoji softwaru, ale výsledky ukazují že to není tak jednoduché jak se čekalo. Randomizovaná studie METR (METR, 2025) na 246 úlohách ukázala, že s AI nástroji byli vývojáři o 19 % pomalejší. Agent často produkuje funkční výstup, nedodržuje přitom vývojové praktiky: slučuje nesouvisející změny, přeskakuje testy, ignoruje specifikaci.

Otázka není jen jestli agent úkol vyřešil, ale jak. Existující benchmarky jako SWE-bench (Jimenez et al., 2024) měří pouze výsledek: vyřešil agent úkol, ano nebo ne. Kvalitu práce, transparentnost procesu a možnost řídit chování agenta v průběhu vývoje tyto benchmarky nepostihují. Agent může hlásit úkol jako dokončený, přestože implementace pokrývá jen část požadavků, a bez metrik procesu to není jak systematicky odhalit.

Chování agenta lze přizpůsobit trénováním modelu (fine-tuning) nebo instrukcemi v kontextovém okně (in-context learning). Shin et al. (Shin et al., 2025) ukazují, že automatizované prompty systematicky nepřekonávají fine-tuned modely, ale iterativní zpřesňování instrukcí výrazně zlepšuje výsledky na úlohách s kódem. S rostoucí velikostí kontextových oken se instrukce stávají praktičtější alternativou: jsou dostupné komukoliv, iterativně upravitelné a neomezují emergentní schopnosti modelu. Pro trénování modelů existují zavedené postupy; pro systematické navrhování a vyhodnocování instrukcí obdobný postup dosud nevznikl.

Tato práce navrhuje evaluační systém který měří nejen výsledek, ale i proces a kvalitu práce agenta, a popisuje postup jak s jeho pomocí iterativně vylepšovat instrukce.

1.2 Cíle práce

[DRAFT]

1. Navrhnout sadu metrik která měří proces a kvalitu práce AI agenta, ne jen výsledek.
2. Iterativním postupem navrhnout instrukce které dovedou agenta k dodržování stanovených exit kritérií.
3. Ablacemi identifikovat, které složky instrukční sady přispívají k měřenému chování agenta a které jsou redundantní.

1.3 Rozsah práce

[DRAFT]

Evaluační systém a postup pro návrh instrukcí nelze navrhnout čistě teoreticky. Je potřeba je ověřit v praxi: ukázat, že metriky skutečně zachytí rozdíly v chování agenta, když dostane jiné instrukce, a že iterativní postup vede ke zlepšení. K tomu slouží případová studie, ve které opakovaně pouštíme agenta s různými variantami instrukcí a pokaždé měříme, co se změnilo.

Aby bylo možné vliv instrukcí měřit, musí být všechno ostatní stejné: model, nástroje, specifikace i výchozí stav repozitáře. Jedinou proměnnou jsou instrukce které agent dostane. Kdyby se měnilo víc věcí najednou, nebylo by jasné, co způsobilo změnu v chování.

Případová studie probíhá na jednom projektu: systému upomínek faktur. Jde o projekt s deterministickou logikou, kde stejný vstup vždy dá stejný výstup. To je důležité, protože výsledky lze ověřit automatizovanými testy, ne subjektivním posouzením. Agent pracuje od specifikace k implementaci, tedy od požadavků přes návrh po funkční kód a testy.

Práce neporovnává různé modely ani programovací jazyky a neporovnává agenta s lidským vývojářem. Navržený evaluační systém a iterativní postup jsou koncipovány obecně: kdokoli je může vzít a použít na svém projektu s vlastními prahy. Konkrétní instrukce a naměřené hodnoty platí pro tuto případovou studii. Zdůvodnění volby projektu a podrobnosti experimentálního designu popisuje kapitola 3.

2. Teoretická východiska

2.1 Softwarové inženýrství a životní cyklus

2.1.1 Definice a komplexita software

2.1.2 Fáze životního cyklu

2.1.3 Modely a metodiky

2.1.4 Měření kvality software

Klasifikace softwarových metrik

Testování a mutation testing

Statická analýza a složitost kódu

LLM-as-judge

2.2 AI coding agenti a scaffolding

2.2.1 Základní pojmy

2.2.2 Typy coding agentů

2.2.3 Jak agenti mění SDLC

2.2.4 Problém kontextu

2.2.5 Přístupy k memory a kontextu

2.2.6 Struktura instrukcí

2.3 Evaluace agentů a instrukcí

2.3.1 Hodnocení agentů a benchmarky

2.3.2 Evaluace instrukcí

3. Metodika

3.1 Návrhový výzkum (DSR)

[DRAFT]

Kapitola 1 definuje tři cíle (navrhnout sadu metrik, iterativním postupem navrhnout instrukce a ablacemi identifikovat které složky přispívají), jejichž naplnění vyžaduje navrhovat řešení, ověřovat ho v praxi a na základě výsledků iterativně upravovat instrukce. Takový postup, kdy se navrhuje řešení a opakovaně vyhodnocuje, odpovídá návrhovému výzkumu (Design Science Research, DSR). Hevner et al. (Hevner et al., 2004) ho definují jako iterativní cyklus návrhu a vyhodnocení: řešení se navrhne, vyzkouší, a na základě výsledků upraví. Peffers et al. (Peffers et al., 2008) tento cyklus formalizují do šesti aktivit: (1) identifikace problému, (2) definice cílů řešení, (3) návrh, (4) demonstrace, (5) vyhodnocení a (6) komunikace výsledků. V naší práci odpovídá identifikace problému a cíle kapitole 1, návrh a demonstrace probíhají formou případové studie (tato kapitola a kapitola 4), vyhodnocení shrnuje kapitola 5 a komunikací je samotný text práce.

Aktivity návrh, demonstrace a vyhodnocení se v DSR opakují dokud řešení nedosáhne požadované kvality. V naší práci operacionalizujeme tento cyklus jako čtyři kroky: Spuštění (spustit agenta), Měření (změřit výstup), Diagnóza (identifikovat příčiny) a Úprava (upravit instrukce). Každý průchod těmito kroky odpovídá jednomu průchodu Peffersovými aktivitami návrh → demonstrace → vyhodnocení. Podrobný popis kroků uvádí sekce 3.3.4. Tento postup má dvě fáze: pilotní iterace, kde cyklus opakujeme dokud agent nesplní stanovená kritéria, a komparativní variace, kde z fungujících instrukcí systematicky odebíráme nebo nahrazujeme jednotlivé části a měříme dopad na chování agenta.

DSR vyžaduje demonstraci a vyhodnocení navrženého řešení. Hevner et al. (Hevner et al., 2004) uvádí několik možných přístupů: řízený experiment (kontrola proměnných, statistická analýza), simulace, nebo případová studie. Řízený experiment by vyžadoval dostatečný počet běhů pro statistickou sílu, což je při ceně jednoho běhu (tisíce tokenů, desítky minut) nepraktické. Simulace nepřipadá v úvahu, protože chování agenta nelze modelovat bez skutečného spuštění. Zvolili jsme tedy případovou studii na jednom projektu (systém upomínek faktur), která umožňuje opakované běhy s různými instrukcemi a podrobnou analýzu každého běhu.

Z jednoho projektu nelze tvrdit, že výsledky platí pro všechny typy software. Cílem je ukázat, že navržená sada metrik dokáže kvantifikovat chování agenta a že iterativní postup vede k měřitelnému zlepšení. Yin (R. K. Yin, 2018) pro tento typ výzkumu používá pojem analytická generalizace: neříkáme “takhle to funguje vždy”, ale “na tomto případě jsme ukázali princip, který lze ověřit na dalších projektech”. Yin tento design nazývá embedded single-case: zkoumáme jeden projekt, ale v rámci něj provádíme více běhů agenta. Každý běh má vlastní instrukce, vlastní git historii a vlastní sadu naměřených metrik, takže je můžeme porovnávat

mezi sebou.

3.2 Sada metrik

[DRAFT]

Každý experimentální běh probíhá tak, že agent dostane prázdné GitHub repo, specifikaci v Issue #1 a instrukce v `AGENTS.md`, a autonomně implementuje zadaný projekt. Výstup jednoho běhu (kód, testy, git historie) měříme sadou 19 metrik. Postup běhu a experimentální design popisuje sekce 3.3.

Sada metrik pokrývá tři otázky: jak agent pracoval, co vyrobil a za jakou cenu. Fenton a Bieman (Fenton & Bieman, 2014) klasifikují měřitelné entity v softwarovém inženýrství do tří kategorií: proces, produkt a zdroj. V kontextu AI agentů dospívají k obdobnému rozdělení Yin et al. (2025) (Z. Yin et al., 2025), kteří hodnotí agentní frameworky ve třech dimenzích: effectiveness (výsledek), efficiency (průběh práce) a overhead (spotřebované zdroje). Naše sada metrik sleduje stejnou strukturu a obsahuje 19 metrik ve třech kategoriích:

- P – proces (P1–P8)** Dodržel agent předepsaný postup? Měříme dodržování workflow pravidel z instrukcí (P1–P5) a kvalitu procesních výstupů jako commit messages, issue a PR descriptions (P6–P8).
- Q – kvalita produktu (Q1–Q8)** Je kód kvalitní? Měříme funkční korektnost (Q1–Q2), kvalitu testů (Q3–Q4) a udržitelnost kódu (Q5–Q8).
- E – efektivita (E1–E3)** Za jakou cenu? Zaznamenáváme spotřebu tokenů (E1), čas (E2) a míru autonomie (E3).

Výběr konkrétních metrik vychází z potřeb případové studie a dostupných nástrojů; sada si neklade nárok na úplnost. Teoretické základy jednotlivých metrik (proč jsou validní, jaká je evidence) popisuje sekce 2.1.4. Tato sekce se zaměřuje na *jak* konkrétně měříme: jaký nástroj, s jakou konfigurací, co je vstup a co výstup. U každé metriky uvádíme v krátkosti co měří a pak podrobně jak.

3.2.1 Procesní metriky (P1–P8)

[DRAFT]

Procesní metriky (Fenton & Bieman, 2014) měří *jak* agent pracuje. Instrukce v `AGENTS.md` definují strukturovaný vývojový postup: agent má odvozovat práci ze specifikace, organizovat ji přes issues a branches, psát testy před implementací a dokumentovat rozhodnutí v commit messages a PR descriptions.

P1–P5: compliance (binární)

P1 (issues before code) ověřuje, zda agent vytvořil issues s timestampem před prvním kódovým commitem. Měříme porovnáním `created_at` nejstaršího agentova issue (GitHub API) s časem prvního commitu obsahujícího soubory v `src/` nebo `tests/`.

P2 (branch per issue) ověřuje, zda agent vytvořil pro každé issue vlastní větev. Měříme počet remote branches (bez `main`) vůči počtu issues: podmínka $branches \geq issues$ je splněna tehdy, nenastalo-li slučování více issues do jedné větve.

P3 (test-first commits) ověřuje, zda agent psal testy před implementací. Primární indikátor: existuje alespoň jeden commit s prefixem `test:` a zároveň alespoň jeden `feat:`. Přesnější měření poskytuje behavioral trace (sekce 3.3.3): počet větví, kde první zápis do `src/` předchází prvnímu zápisu do `tests/` (`tddOrderViolations`).

P4 (PRs linked to issues) ověřuje, zda každý PR obsahuje odkaz na issue. GitHub API vrací tělo PR; podmínka: všechna PR těla obsahují regex `Closes #N`.

P5 (no existing test modifications) ověřuje, že agent nepřepisoval již existující testové soubory. Metrika zachycuje `modified` změny (`--diff-filter=M`) na testovacích souborech, tedy situaci kdy agent dříve vytvořený test později upraví místo toho, aby opravil implementaci. Nově přidané testy se do P5 nezapočítávají.

P6–P8: kvalita procesních artefaktů (LLM-as-judge)

P6 (commit message quality) hodnotí popisnost commit messages: atomicitu, konvenční prefix a srozumitelnost co a proč bylo změněno. Hodnocení provádí LLM-as-judge na škále 1–3 (sekce 3.2.4).

P7 (issue description quality) hodnotí popisnost issue descriptions: jasnost scope, přítomnost acceptance criteria, dostatečnost pro implementaci. Hodnocení provádí LLM-as-judge na škále 1–3.

P8 (PR description quality) hodnotí popisnost PR descriptions: přítomnost odkazu na issue, popis co a proč, dostatečnost pro code review. Hodnocení provádí LLM-as-judge na škále 1–3.

3.2.2 Produktové metriky (Q1–Q8)

[DRAFT]

Produktové metriky (Fenton & Bieman, 2014) měří *co* agent vyrobil. Pokrývají tři oblasti: funguje implementace správně (Q1–Q2), detekují agentovy testy skutečné chyby (Q3–Q4) a je kód udržitelný (Q5–Q8).

Funkční korektnost (Q1–Q2)

Q1 (API contract match) ověřuje, zda agentův kód dodržuje definované rozhraní. Referenční typy se importují a zkompilují proti agentovu kódu (`tsc`). Výsledek je binární: typy sedí, nebo ne. Q1 je vstupní podmínkou pro Q2: pokud agentův kód neimplementuje správné API, referenční testy nelze ani zkompilovat, a výsledek Q2 by byl nesmyslný.

Q2 (referenční test pass rate) měří kolik z referenčních testů projde na agentově implementaci. Vitest spustí 42 testů proti agentovu kódu; výsledek je počet passing testů z 42. Testy ověřují chování přes veřejné API (black-box, sekce 2.1.4), takže je lze spustit na libovolném běhu nezávisle na interní struktuře. Konstrukce referenční test suite popisuje sekce 4.1.2.

Kvalita testů (Q3–Q4)

Q3 (mutation score) měří, jestli agentovy testy skutečně detekují chyby (sekce 2.1.4). Stryker s konfigurací `--mutate 'src/**/*.ts'` a mutátory pro TypeScript (conditional, arithmetic, string, logical operators) systematicky zavádí drobné změny do zdrojového kódu. Vstup: agentovy testy + zdrojový kód. Výstup: procento zabitých mutantů z celkového počtu.

Q4 (AC coverage) měří, kolik z 25 acceptance criteria má odpovídající test v agentově test suite. Na rozdíl od Q2 (funguje implementace?) se Q4 ptá, jestli agent *testoval všechno co měl*. Judge dostane seznam 25 AC ze specifikace a agentovu test suite. Pro každé AC určí, zda existuje test který ho pokrývá. Výstup: počet pokrytých AC z 25. Hodnocení provádí LLM-as-judge (sekce 3.2.4).

Kvalita kódu (Q5–Q8)

Q5 (lint warnings) počítá varování a chyby z ESLint s fixní konfigurací (pravidla: recommended + strict TypeScript rules). Konfigurace je součástí experimentální infrastruktury, ne agentova repo — agent ji nemůže měnit. Výstup: celkový počet warnings + errors.

Q6 (typecheck errors) počítá chyby z `tsc --noEmit` ve strict mode. Doplnjuje počet explicitních `any` ve zdrojových souborech (`grep` v `src/**/*.ts`) jako proxy pro obcházení typového systému.

Q7 (složitost kódu) měří cyklomatickou složitost per funkce. ESLint complexity rule reportuje funkce překračující nastavený práh. Výstup: maximální složitost across all functions. Prah ≤ 10 per funkce vychází z McCabe (sekce 2.1.4).

Q8 (design quality) hodnotí aspekty které automatizované nástroje nezachytí: pojmenování, oddělení zodpovědností, idiomatický TypeScript, kvalita dokumentace a zbytečná komplexita. Hodnocení provádí LLM-as-judge (sekce 3.2.4). Celkové skóre Q8 je *minimum* pěti

dimenzí, nikoliv průměr. Volba minima je záměrná: jeden slabý rozměr má stáhnout celkový výsledek dolů, aby slabiny nebyly maskované silnými dimenzemi.

3.2.3 Metriky efektivity (E1–E3)

[DRAFT]

Metriky efektivity měří zdroje spotřebované při vývoji. Nemají exit kritérium a slouží k porovnání nákladů mezi iteracemi.

E1 (tokeny) počítá vstupní a výstupní tokeny z exportu agentní session. Sledování E1 motivuje zjištění Gloaguen et al. (Gloaguen et al., 2025), že context files zvyšují inference cost o více než 20 %.

E2 (trvání) měří wall-clock time v minutách od spuštění agenta po poslední commit.

E3 (dokončení) zaznamenává, zda agent dokončil úkol bez pádu, a počet restartů auto-continue pluginu jako proxy pro autonomii.

Se dvěma běhy per variaci jsou tyto hodnoty deskriptivní, ne inferenční.

3.2.4 Aplikace LLM-as-judge

[DRAFT]

Pět metrik (P6, P7, P8, Q4, Q8) hodnotí vlastnosti které nelze extrahovat automatizovaným nástrojem. Pro jejich hodnocení používáme metodu LLM-as-judge (sekce 2.1.4).

Volba modelu. Jako judge používáme GLM-5 (Zhipu AI). Agentní běhy provádí model Minimax — judge je tedy z jiné modelové rodiny. Důvod: model má tendenci hodnotit výstupy svého vlastního druhu lépe, i když jsou objektivně slabší (self-preference bias (Panickssery et al., 2024)). Volba judge z jiné modelové rodiny než agent tento bias snižuje (Verga et al., 2024).

Škála. Hodnocení probíhá na škále 1–3 per dimenze: 1 = nevyhovující, 2 = přijatelné, 3 = dobré. Jemnější škály (1–5, 1–10) produkují při malém počtu hodnocených artefaktů nižší shodu mezi hodnotiteli (Zheng et al., 2023); třístupňová škála je pro tento rozsah spolehlivější.

Rubrika. Judge dostane artefakty agentova běhu (zdrojový kód, commit messages, issue a PR descriptions) spolu s rubrikou obsahující popis každého stupně. Prompt je fixní across all runs, aby hodnocení bylo srovnatelné mezi iteracemi. Rubriky jsou uvedeny v příloze.

Interpretace výsledků. LLM-as-judge zde neslouží jako plně objektivní měření, ale jako strukturované posouzení vlastností, které nelze spolehlivě vyhodnotit deterministickým skrip-

tem. Výsledky těchto metrik proto interpretujeme jako podpůrné kvalitativní indikátory a ne jako hlavní důkazní osu experimentu.

3.3 Experimentální design

3.3.1 Výběr projektu

[DRAFT]

Případová studie potřebuje projekt, na kterém lze opakovaně spouštět agenta s různými instrukcemi a objektivně měřit výsledky. Projekt musí mít deterministickou logiku (stejný vstup vždy dá stejný výstup), aby bylo možné ověřit korektnost automatizovanými testy a mutation testingem. Zároveň musí být dostatečně malý pro více experimentálních běhů, ale obsahovat reálné nuance (hraniční případy, doménová pravidla), aby měl agent co řešit.

Zvolili jsme systém upomínek faktur: systém pro automatické odesílání připomínek k nezaplaceným fakturám. Obsahuje stavový automat (nová, po splatnosti, upomínaná, eskalovaná), časové výpočty (pracovní dny, ochranné lhůty) a pravidla pro eskalaci. Specifikace definuje 25 acceptance criteria, API kontrakt (TypeScript typy a signatury) a doménový slovník. Tato kombinace deterministické logiky a hraničních případů (víkendy, svátky, grace periods) umožňuje objektivně testovat jak funkční korektnost (Q1, Q2), tak kvalitu testů (Q3, Q4).

3.3.2 Fixní proměnné

[DRAFT]

Aby bylo možné měřit vliv instrukcí, musí být všechno ostatní konstantní. Jedinou proměnnou mezi běhy je obsah souboru `AGENTS.md`, procedurální instrukční sady, která definuje pracovní postup, omezení a quality gates. Na rozdíl od generických context files, které typicky popisují adresářovou strukturu a build příkazy (Gloaguen et al., 2025), jde o fokusované instrukce s konkrétními procedurami (Li et al., 2025).

Fixní proměnné jsou:

Prázdné GitHub repo obsahuje pouze `AGENTS.md`, konfiguraci agenta a auto-continue plugin. Žádný existující kód, testy ani `package.json`. Agent musí inicializovat projekt a zvolit strukturu sám.

Specifikace v GitHub Issue #1 obsahuje 25 acceptance criteria, API kontrakt (TypeScript typy a signatury), doménový slovník a out of scope. Jediný vstup který agent dostane kromě instrukcí.

Auto-continue plugin je hook který detekuje kdy se agent zastaví a automaticky ho re-

startuje. Počítadlo restartů a kontrola otevřených issues zajišťují běh bez manuálního zásahu. Metrika E3 z toho čerpá.

Model MiniMax-M2.5 byl zvolen jako model schopný autonomně dokončit zadaný úkol. Cílem práce je vyhodnotit sadu metrik a iterativní postup, ne srovnávat výkon modelů — volba konkrétního modelu je v tomto ohledu sekundární. MiniMax-M2.5 je zároveň z jiné modelové rodiny než judge (GLM-5, Zhipu AI), čímž se eliminuje self-preference bias (sekce 2.1.4).

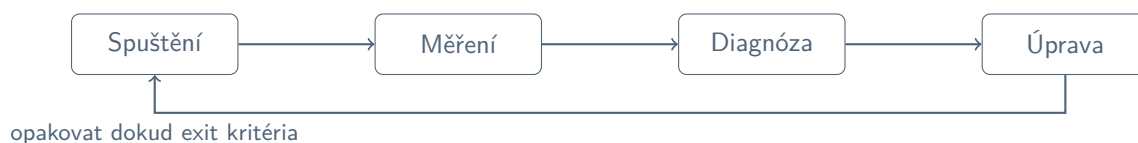
Běhové prostředí (Docker container) zajišťuje, že agent nemá přístup ke globální konfiguraci nástroje, MCP serverům ani rozšířením nainstalovaným na hostitelském systému. Izolace zaručuje, že chování agenta je determinováno výhradně obsahem `AGENTS.md` a specifikací — ne vedlejšími vstupy z prostředí. Sdílený autentizační svazek zajišťuje konzistentní přístup k API napříč běhy.

System prompt agenta (`build.md`) nahrazuje výchozí system prompt nástroje OpenCode, jehož instrukce o verzování konfliktvaly s procesními požadavky experimentu. Vlastní system prompt obsahuje pouze obecné kódové konvence (styl, bezpečnost, zákaz komentářů) a odkaz na `AGENTS.md`. Veškeré procesní instrukce jsou výhradně v `AGENTS.md`, aby procesní chování agenta bylo řízeno jedinou proměnnou experimentu.

3.3.3 Iterativní cyklus

[DRAFT]

Pilotní fáze postupuje opakováním čtyřkrokového cyklu, dokud agent nesplní exit kritéria. Každý průchod cyklem je jedna experimentální iterace — vstupem je aktuální verze `AGENTS.md`, výstupem upravená verze pro příští běh.



Spuštění. Skript `new-run.ts` připraví izolované prostředí: vytvoří prázdné GitHub repo, zkopíruje aktuální `AGENTS.md`, vytvoří Issue #1 se specifikací a spustí agenta v Docker kontejneru. Agent dostane výhradně prázdné repo, instrukce a specifikaci — žádný existující kód ani závislosti. Po dokončení se exportuje session transcript (`transcript.json`).

Měření. Skript `analyze-run.ts` extrahuje deterministické metriky automaticky: git log a GitHub API (P1), výstup Vitest a Stryker (Q2, Q3), statická analýza (Q5–Q7), session metadata (E1–E3). Kvalitativní metriky (P6, P7, P8, Q4, Q8) hodnotí LLM-as-judge GLM-5 podle fixní rubriky (sekce 3.2.4). Výstupem je `FINDINGS.md` s tabulkou metrik a faktickým popisem chování agenta.

Diagnóza. Z naměřených hodnot a behaviorálního popisu se identifikuje kde a proč agent nedodržel očekávané chování. Analýza postupuje podle čtyř rámců:

- Mao et al. (Mao et al., 2025) — která složka instrukcí selhala a proč
- Hassan et al. (Hassan et al., 2025) — zda obsah vyvažuje proceduru, kontext a omezení
- Lulla et al. (Lulla et al., 2026) — zda každý řádek přidává hodnotu
- Razavi, Breunig (Breunig, 2025; Razavi & Fard, 2025) — proč agent instrukci opakovaně ignoruje

Výstupem je `DIAGNOSIS.md` s identifikovanými příčinami a návrhem změn.

Úprava. Na základě diagnózy se upraví `AGENTS.md`. Každá změna odpovídá jednomu identifikovanému problému, je podložena citací a zaznamenána v changelogu — aby bylo zpětně dohledatelné proč daná úprava vznikla a co měla opravit. Přestrukturování instrukcí je preferováno před jejich rozšiřováním: redundantní obsah zvyšuje inference cost bez přínosu k úspěšnosti (Gloaguen et al., 2025) a fokusované instrukce překonávají vyčerpávající dokumentaci (Li et al., 2025).

3.3.4 Pilotní fáze

[DRAFT]

Pilotní fáze běží dokud agent nesplní exit kritéria ze sloupce *Exit kritérium* v tabulce 3.1. Kritéria mají dva typy: deterministická (P1–P5, Q1–Q2, Q4–Q6) mají přirozenou binární hranici — buď splněno nebo ne; minimální standard (P6–P8, Q3, Q7–Q8) je práh zdůvodněný literaturou. E1–E3 jsou záznamové metriky bez kritéria, slouží k porovnání efektivity mezi běhy.

S ohledem na scope práce (jeden projekt, jeden model) stačí jeden úspěšný běh bez manuálního zásahu. Každá iterace produkuje aktualizovaný `AGENTS.md`, záznam změn se zdůvodněním a kompletní P/Q/E metriky.

3.3.5 Komparativní variace

[DRAFT]

Z fungující sady instrukcí (výstup pilotní fáze) systematicky měníme jednotlivé části a měříme dopad na chování agenta. Používáme dva typy změn: ablace (odebrání části, tedy funguje agent i bez ní?) a substituce (nahrazení alternativní formulací, tedy která varianta produkuje lepší výsledky?).

Konkrétní variace nelze specifikovat předem. Pilotní fáze *generuje* hypotézy o tom, které složky instrukcí jsou pro chování agenta podstatné, a komparativní fáze je testuje. Výběr variací bude vycházet z diagnostiky pilotních běhů pomocí rámců popsanych v sekci 3.3.4.

Například pokud se v pilotu ukáže, že agent konzistentně ignoruje pravidlo o test-first commitech, komparativní variace může testovat, zda odebrání tohoto pravidla změní i ostatní metriky, nebo zda alternativní formulace vede k lepšímu dodržování.

Se dvěma běhy per variaci nelze statisticky odlišit efekt změny od šumu způsobeného neterminismem LLM (sekce 3.5). Výsledky komparativní fáze proto interpretujeme jako indikativní, ne kauzální.

3.4 Přehledová tabulka

[DRAFT]

Tabulka 3.1 shrnuje všechny metriky na jednom místě. Sloupec *typ* rozlišuje deterministická exit kritéria (přirozená binární hranice), minimální standard (práh zdůvodněný literaturou) a záznamové metriky (bez kritéria, slouží k porovnání mezi běhy). Automatizované metriky (P1–P5, Q1–Q3, Q5–Q7, E1–E3) sbírá jeden skript; kvalitativní metriky (P6–P8, Q4, Q8) hodnotí LLM-as-judge (sekce 3.2.4).

Tabulka 3.1: Sada metrik: kód, měřená vlastnost, nástroj, exit kritérium a typ

Kód	Metrika	Nástroj	Exit kritérium	Typ
<i>Procesní (P) — jak agent pracuje</i>				
P1	Issues before code	git, GitHub API	pass	deter.
P2	Branch per issue	git, GitHub API	pass	deter.
P3	Test-first commits	git log	pass	deter.
P4	PRs linked to issues	GitHub API	pass	deter.
P5	No existing test modifications	git diff	pass	deter.
P6	Commit message quality	LLM-as-judge (GLM-5)	$\geq 2/3$	min.
P7	Issue description quality	LLM-as-judge (GLM-5)	$\geq 2/3$	min.
P8	PR description quality	LLM-as-judge (GLM-5)	$\geq 2/3$	min.
<i>Produktové (Q) — co agent vyrobil</i>				
Q1	API contract match	tsc (import + typecheck)	match	deter.
Q2	Referenční test pass rate	Vitest (42 testů)	42/42	deter.
Q3	Mutation score	Stryker	$\geq 70\%$	min.
Q4	AC coverage agentových testů (25 krit.)	LLM-as-judge (GLM-5)	25/25	deter.
Q5	Lint warnings	ESLint	0	deter.
Q6	Typecheck errors	tsc --noEmit	0	deter.
Q7	Cyklomatická složitost	ESLint (complexity)	$\leq 10/\text{fn}$	min.
Q8	Design quality	LLM-as-judge (GLM-5)	$\geq 2/3$	min.
<i>Efektivita (E) — za jakou cenu</i>				
E1	Tokeny (input + output)	OpenCode export	—	záznam
E2	Trvání (minuty)	session timestamps	—	záznam
E3	Dokončení + restarts	auto-continue log	—	záznam

3.5 Omezení a validita

[DRAFT]

Každý výzkumný design má omezení. Následující analýza vychází z kategorizace Runeson a Höst (Runeson & Höst, 2009).

Konstruktová validita (měříme to co chceme měřit?). Jednotlivé metriky vychází z existující teorie: mutation testing (Papadakis et al., 2019), cyklomatická složitost (McCabe, 1976), taxonomie procesních a produktových metrik (Fenton & Bieman, 2014). Konkrétní kombinace metrik do sady a volba exit kritérií jsou autorské a představují návrh, ne ověřený standard. Pět metrik (P6, P7, P8, Q4, Q8) obsahují subjektivní složku (LLM-as-judge). Tyto metriky proto slouží jako podpůrné kvalitativní indikátory, zatímco hlavní opora vyhodnocení stojí na deterministických metrikách a auditovatelných artefaktech. Spolehlivost LLM-as-judge nebyla validována proti lidskému hodnocení (např. Cohenovým κ) z důvodu časových omezení; výsledky kvalitativních metrik proto nelze interpretovat jako objektivní měření. Experiment rozlišuje dva druhy testů: referenční testy (měřené metrikou Q2, spouštěné po běhu skriptem, agentovi neviditelné) a agentovy vlastní testy (psané agentem během běhu, viditelné a spustitelné agentem). Metrika P5 se vztahuje výhradně k agentovým vlastním testům. Referenční testy jsou odvozeny ze specifikace metodou TDD (Mathews & Nagappan, 2024), čímž se snižuje riziko test oracle problému. Při validaci měřicí infrastruktury byly identifikovány známé nedostatky (Q4, P7, Q6) a změny prostředí mezi běhy (Docker izolace, E1 neporovnatelnost, temperature); tyto poznatky jsou konzistentní napříč běhy a jsou diskutovány v sekci 5.3.

Interní validita (jsou závěry podložené daty?). Hlavní hrozba je nedeterminismus LLM: stejné instrukce mohou při různých bězích dát různé výsledky (Razavi & Fard, 2025). Dva běhy per variaci jsou mitigací, ale neposkytují statistickou sílu. Další hrozbou je confirmation bias: autor navrhuje instrukce i evaluuje výsledky. Tuto hrozbu oslabují automatizované metriky (P1, Q1–Q3, Q5–Q7), které nepodléhají subjektivnímu posouzení, a volba judge modelu z jiné rodiny. Mitigací je také auditovatelný řetězec rozhodnutí (changelog per iterace, korekční poznámky v FINDINGS.md) a oddělení měřicí infrastruktury od agentova prostředí. Jak se tyto hrozby projeví v praxi — včetně diagnostické chyby výzkumníka, změn prostředí mezi běhy, nenastavené temperature modelu a tichých ukončení nástroje — diskutuje sekce 5.3.

Externí validita (lze zobecnit?). Případová studie na jednom projektu, jednom modelu a jednom agentním nástroji neumožňuje statistickou generalizaci. Yin (R. K. Yin, 2018) pro tento typ výzkumu rozlišuje analytickou generalizaci: z jednoho případu lze ukázat principy, ne statistické zákonitosti. Systém upomínek má deterministickou logiku a výsledky nemusí platit pro projekty s uživatelským rozhraním, strojovým učením nebo nedeterministickými výstupy. Přenositelné jsou metriky a postup (kdokoliv je může použít na svém projektu), konkrétní naměřené hodnoty a instrukce platí pro tuto studii.

Výzkum nezahrnuje lidské účastníky. Veškerá data (git log, metriky, session transcripts) jsou

generována experimentální infrastrukturou. Zdrojový kód a experimentální data jsou veřejně dostupná.

4. Praktická část

[DRAFT]

Tato kapitola popisuje provedení případové studie. Struktura sleduje chronologii experimentu: nejprve příprava tří výchozích podkladů (specifikace, referenční implementace, instrukce), pak pilotní iterace kde se instrukce opakovaně upravují na základě naměřených metrik, a nakonec komparativní variace kde z fungující sady systematicky odebíráme jednotlivé části a měříme dopad.

4.1 Příprava experimentu

[DRAFT]

Před spuštěním pilotních běhů bylo nutné připravit tři výchozí podklady každého experimentálního běhu: specifikaci projektu (co má agent implementovat), referenční implementaci (měřicí nástroj pro Q2 – referenční test pass rate) a baseline instrukce (jak má agent pracovat).

4.1.1 Konstrukce specifikace

[DRAFT]

Specifikace definuje *co* má agent implementovat. V této práci má formu strukturované GitHub issue (Issue #1), kterou agent dostává jako zadání. Tato volba navazuje na metodické vymezení fixních vstupů v sekci 3.3: agent kromě instrukcí v souboru `AGENTS.md` pracuje právě s touto specifikací a žádný další architektonický návrh nedostává. Issue se skládá ze dvou částí: **Requirements** (co business potřebuje) a **API Contract** (co musí implementace exportovat). Behavioral model, state diagram ani explicitně formulované invarianty nejsou součástí zadání; návrh interní architektury je ponechán agentovi. API kontrakt, tedy exportované funkce a typy, představuje jediné technické omezení implementace a současně slouží jako vstup pro metriku **Q1** (API contract match).

Podoba specifikace vychází ze tří principů důležitých pro tuto evaluační úlohu: jednoznačnosti požadavků, jejich testovatelnosti a omezení interpretační volnosti agenta. Specifikace je proto záměrně stručná a strukturovaná. Stručnost reaguje na poznatek, že delší kontext může zhoršovat schopnost jazykového modelu spolehlivě využít relevantní informaci (Liu et al., 2024). Současně byla zvolena taková struktura, která minimalizuje redundantní instrukce a udržuje přímou vazbu mezi požadavkem, implementací a následným vyhodnocením (Bockeler, 2025).

Specifikace obsahuje:

- **25 acceptance criteria** ve formátu Given/When/Then s konkrétními hodnotami — eskalační flow (8 stavových přechodů), platby, terminální stavy, pause/resume, manuální advance, konfigurovatelné timeouty, výpočet pracovních dní
- **API kontrakt** v TypeScriptu — veřejná funkce `process(state, event, now)` vracějící nový stav a action descriptor; typy pro 12 stavů, 6 událostí, 3 typy akcí; konfigurační interface pro timeouty a svátky
- **Doménový slovník** — 9 pojmů definujících business doménu (dunning, grace period, business days, action descriptor aj.)
- **Out of scope** — 6 explicitně vyloučených oblastí (payment retry, late fees, partial payments, email sending, scheduling, persistence)

Formát acceptance criteria přímo ovlivňuje měřitelnost. Struktura Given/When/Then odděluje výchozí stav, podnět a očekávaný výsledek, což usnadňuje převod jednotlivých požadavků do testovacích scénářů. V této práci z ní proto přímo vycházejí referenční testy (metrika **Q2**) i mapování acceptance criteria na agentem vytvořené testy (metrika **Q4**). Zvolený formát tak neslouží jen k zápisu business požadavků, ale i k jejich konzistentnímu vyhodnocení v rámci experimentu.

Specifikace definuje systém jako pure function: `process(state, event, now)` vrací nový stav a action descriptor místo provádění side effects. Toto rozhodnutí bylo přijato proto, aby bylo možné nad jedním deterministickým rozhraním konzistentně vyhodnocovat referenční testy (metrika **Q2**) i mutation testing (metrika **Q3**). Pevné TypeScript signatury současně zajišťují, že stejné testovací prostředí lze použít napříč jednotlivými běhy nezávisle na interní struktuře agentem vytvořeného řešení.

Specifikace obsahuje explicitní sekci out of scope, která vymezuje 6 oblastí mimo zadání. V kontextu této studie tato sekce nevymezuje jen hranice implementace, ale také omezuje riziko, že agent rozšíří řešení nad rámec sledovaného zadání a zkomplikuje tak interpretaci výsledků.

4.1.2 Referenční implementace

[DRAFT]

Referenční implementace slouží jako měřicí nástroj: definuje strop kvality proti kterému se porovnávají agentovy běhy. Její role v metrikách popisuje sekce 3.2.2.

Testy byly napsány metodou *spec-first TDD* (sekce 2.1.4): expected values pochází ze specifikace, ne z pozorování kódu.

Výsledná implementace obsahuje 42 behavioral testů pokrývajících všech 25 acceptance criteria (některá kritéria vyžadují více testových případů pro hraniční situace), dosahuje mutation score 91,9 %. Kód kompiluje bez typecheck errors, neobsahuje žádné `any` a má 3 lint warnings (kosmetické, bez vlivu na funkční korektnost). LLM-as-judge (**Q8**) ohodnotil design quality

na 3/3 ve všech dimenzích. Tyto výsledky validují, že specifikace je implementovatelná a že exit kritéria (sekce 3.3.4) jsou dosažitelná lidským vývojářem.

4.1.3 Konstrukce baseline instrukcí

[DRAFT]

Baseline verze `AGENTS.md` byla konstruována čistě z literatury. Cílem bylo vytvořit výchozí bod jehož každá komponenta má opodstatnění v empirických zjištěních.

Konstrukce proběhla ve třech krocích: smazání předchozí ad-hoc verze, návrh struktury podle pořadí komponent, které Mao et al. (Mao et al., 2025) zjistili v analýze 2 163 produkčních šablon, a mapování tří dimenzí chování (sekce 3.2) na konkrétní instrukce. Mao identifikují sedm typů komponent (Role, Directive, Context, Workflow, Output, Constraints, Examples); naše baseline sekce (Role, Goal, Specification, Environment, Process, Package Quality, Constraints) jsou adaptací tohoto rámce na doménu případové studie.

Mao et al. zjistili, že v produkčních šablonách se Role a Directive nejčastěji objevují na začátku dokumentu, zatímco Constraints na konci. Toto pozorované pořadí tvoří páteř baseline dokumentu.

Sekce Package Quality definuje očekávání na kvalitu kódu: modulární struktura (typy, business logika a veřejné API v oddělených souborech), striktní typový systém bez obcházení (`any`), JSDoc dokumentace na exportech a jasně definovaný vstupní bod (`index.ts` re-exportuje pouze veřejné API). Tyto požadavky přímo ovlivňují metriky **Q5–Q8** (lint, typecheck, složitost kódu, design quality). Lulla et al. (Lulla et al., 2026) zjistili, že přítomnost instrukčního souboru je spojena s nižší mediánovou dobou běhu (−28,6 %), ačkoli účinnost jednotlivých typů obsahu dosud nebyla izolována.

Procesní sekce obsahuje stručné instrukce pro spec-first TDD (Mathews & Nagappan, 2024), dekompozici do sub-issues, branch-per-issue a conventional commits. Li et al. (Li et al., 2025) ukazují, že procedurální guidance (“jak pracovat”) je efektivnější než popisná dokumentace (“jak vypadá codebase”). Z procesní sekce byly odstraněny bash příklady, protože redundantní obsah zvyšuje inference cost bez benefitu (Gloaguen et al., 2025).

Sekce Constraints obsahuje explicitní zákazy: nekombinovat issues, nemodifikovat již existující testy, nepřepisovat git historii. Mao et al. (Mao et al., 2025) zjistili, že exclusion constraints (46 %) jsou nejčastějším typem omezení v reálných šablonách (46 % výskytů).

Výsledný dokument má 53 řádků a ~350 slov. Tabulka 4.1 ukazuje mapování jednotlivých sekcí na metriky a obrázek 4.1 uvádí kompletní znění.

Baseline AGENTS.md

```
# AGENTS.md --- Dunning System

## Role

You are a senior TypeScript developer building a production-grade npm package. You prioritize clean architecture, spec compliance, and test quality.

## Goal

Implement the dunning system (billing reminder state machine) specified in GitHub Issue #1. Deliver a modular, documented, publishable TypeScript package.

## Specification

Read Issue #1 completely before writing any code. It contains:
- Acceptance Criteria --- every one must be satisfied and tested
- API Contract --- exact TypeScript signatures to export
- Domain Glossary --- use these terms in code and documentation
- Out of Scope --- do not implement

All design decisions must trace back to the spec. Do not invent requirements.

## Environment

- Runtime: Node.js, TypeScript (strict mode, ESM)
- Test runner: Vitest
- Build: tsc
- Tools: GitHub CLI (`gh`), Git

## Process

Decompose the spec into focused GitHub issues before writing code. Work through them sequentially:

1. Create issues --- one concern per issue, starting with project setup
2. For each issue: branch from main, write tests first, implement, PR with "Closes #N", merge
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Run `tsc --noEmit` before every PR --- no type errors allowed

## Package Quality

Structure the code as a production-grade npm package:

- Modular architecture --- separate files for types, business logic, utilities, and public API re-exports. No single-file implementations.
- Strict TypeScript --- no `any`, explicit return types on public API, `readonly` where applicable.
- Documentation --- JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds).
- Clean API surface --- `src/index.ts` re-exports only the public API. Internal modules are not exposed.

## Constraints

- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the spec --- fix the code, not the test.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest, typescript).
```

Obrázek 4.1: Baseline AGENTS.md (pilot-r1) — kompletní znění instrukční sady použité jako výchozí bod pilotních iterací.

Tabulka 4.1: Mapování sekcí baseline `AGENTS.md` na metriky

Cíl	Sekce <code>AGENTS.md</code>	Metriky
Spec-first	Specification, Process krok 2	P1
Strukturovaný workflow	Process kroky 1–4	P2, P3, P4
TDD	Process krok 2, Constraints	P3, P5
Modulární kód	Package Quality	Q5, Q7, Q8
API kompatibilita	Specification (API Contract)	Q1
Funkční korektnost	Specification (AC)	Q2, Q4

Specifikace, referenční implementace a baseline instrukce tvoří výchozí bod experimentu. Následující sekce popisuje průběh pilotních iterací cyklem Spuštění/Měření/Diagnóza/Úprava (sekce 3.3.4).

4.2 Pilotní iterace

[DRAFT]

Každá iterace sleduje cyklus Spuštění/Měření/Diagnóza/Úprava (sekce 3.3.4). U prvního běhu (baseline) uvádíme kompletní tabulku metrik a podrobnou diagnostiku. U dalších běhů popisujeme pouze změny oproti předchozí iteraci; souhrnný přehled všech běhů obsahuje sekce 4.4.

4.2.1 Pilot-r1: baseline

[DRAFT]

Baseline ukázal, že agent zvládne napsat fungující kód, ale nedodržuje předepsaný pracovní postup. První běh s baseline instrukcemi (příloha C), agent Minimax přes OpenCode, jedna session bez restartů. Tabulka 4.2 shrnuje naměřené metriky.

Ze 10 deterministických metrik splnil agent 4 (P1, P4, Q1, Q6).

Chování agenta. Agent vytvořil 11 GitHub issues a rozložil implementaci do 4 pull requestů, každý s vlastní branch. Zvolil modulární strukturu (4 soubory: `types.ts`, `businessDays.ts`, `dunning.ts`, `index.ts`), což odpovídá požadavkům sekce Package Quality v instrukcích. Napsal 59 vlastních testů, všechny procházejí.

Diagnostika. Tři procesní metriky z pěti nebyly splněny: agent sloučil 11 issues do 4 branches (P2), nepoužil oddělené commity pro testy a kód (P3) a upravil vlastní test aby seděl na implementaci (P5). Ve všech třech případech agent vybral rychlejší cestu místo té předepsané.

Tabulka 4.2: Pilot-r1: naměřené metriky

Kód	Metrika	Hodnota	Kritérium	Splněno?
P1	Issues before code	✓	splněno	✓
P2	Branch per issue	×	branches $\geq$ issues	×
P3	Test-first commits	×	test: před feat:	×
P4	PRs linked to issues	✓	všechny PR	✓
P5	Existující testy nezměněny	×	0 změn	×
P6	Commit msg quality	2/3	$\geq 2/3$	✓
P7	Issue quality	2/3	$\geq 2/3$	✓
P8	PR quality	3/3	$\geq 2/3$	✓
Q1	API contract match	match	match	✓
Q2	Ref. test pass rate	39/42	42/42	×
Q3	Mutation score	84 %	≥ 70 %	✓
Q4	AC coverage (agentovy testy)	23/25	25/25	×
Q5	Lint warnings	2	0	×
Q6	Typecheck errors	0	0	✓
Q7	Složitost kódu	2 viol.	0 viol.	×
Q8	Design quality	1/3	$\geq 2/3$	×
E1	Tokeny (in + out)	143k	—	záznam
E2	Trvání	33 min	—	záznam
E3	Dokončení	ano, 0 restartů	—	záznam

Z referenčních testů selhaly 3 z 42 (Q2), převážně v pause/resume logice a výpočtu pracovních dní. Q8 = 1/3 kvůli chybějící JSDoc dokumentaci na veřejném API; ostatní dimenze dosáhly 3/3.

Změna instrukcí pro r2. Čtyři selhání sdílela společný vzorec: instrukce říkala *co* dělat (např. “jedna branch per issue”), ale neukazovala *jak* to provést. Breunig (Breunig, 2025) ukazuje, že když agent pravidlo ignoruje, nepomůže ho zopakovat — je třeba ho přepsat tak, aby se stal součástí pracovního postupu. Razavi a Fard (Razavi & Fard, 2025) dokládají, že drobné změny formulace promptu mohou dramaticky změnit chování modelu, což motivuje cílené přestrukturování místo přidávání dalších pravidel. Každé selhání proto dostalo vlastní opravu:

- P2 (agent kombinoval branches): do pracovního postupu přidán příkaz `git checkout -b issue-N` na začátku každého cyklu — agent tak dostane konkrétní krok místo obecného pravidla
- P3 (agent nepsal testy zvlášť): testy a kód se nově odevzdávají odděleně — nejdřív `git add tests/`, pak `git add src/`
- P5 (agent upravoval vlastní testy): doplněno vysvětlení — pokud test po implementaci selhává, opravit kód, ne test
- Q7 (příliš složité funkce): přidána kontrola `npx eslint src/ --max-warnings 0` před každým pull requestem — agent tak vidí složitost dřív než odevzdá kód

Výřez git diff mezi verzemi AGENTS.md pro běhy r1 a r2 ukazuje rozsah těchto změn.

```
AGENTS.md: pilot-r1 -> pilot-r2

1. Create issues – one concern per issue, starting with project setup
2. For each issue: branch from main, write tests first, implement, PR with "Closes #N", merge
2. For each issue, follow this exact sequence:
  a. `git checkout main && git pull && git checkout -b issue-N` (one branch per issue, no exceptions)
  b. Write tests: `git add tests/ && git commit -m "test: <description>"`
  c. Implement: `git add src/ && git commit -m "feat: <description>"`
  d. Open PR: `gh pr create --title "... " --body "Closes #N"`, merge, delete branch
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Run `tsc --noEmit` before every PR – no type errors allowed
4. Before every PR, run these checks and fix all issues before opening:
  - `tsc --noEmit` – zero type errors
  - `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)

## Package Quality
- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the spec – fix the code, not the test.
- Never modify a test to match your implementation. If a test fails after implementation, fix the code – not the test. Tests encode the spec.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest, typescript).
```

Obrázek 4.2: Vizuální diff změn AGENTS.md mezi pilot-r1 a pilot-r2

4.2.2 Pilot-r2

[DRAFT]

Dvě opravy zabraly (P5, Q7), ale dvě klíčové slabiny přetrvaly (P2, P3) a kvalita kódu se zhoršila (Q2, Q3). Druhý běh ověřoval čtyři změny instrukcí. Tabulka 4.3 ukazuje metriky kde došlo ke změně.

Tabulka 4.3: Pilot-r2: metriky s změnou oproti r1

Kód	Metrika	r1	r2	Trend
P5	Existující testy nezměněny	×	✓	opraveno
Q2	Ref. test pass rate	39/42	32/42	horší
Q3	Mutation score	84 %	68 %	pod prahem
Q5	Lint warnings	2	1	zlepšení
Q7	Složitost kódu	2 viol.	0	opraveno
Q8	Design quality	1/3	1/3	beze změny
E1	Tokeny (in + out)	143k	380k	záznam
E2	Trvání	33 min	37 min	záznam

Diagnostika. P5 a Q7 opraveny, lint warnings sníženy na 1. P2 a P3 přetrvala, ale jinak než v r1: v r1 agent psal kód bez testů, v r2 naopak sloučil testy a kód do jednoho commitu s prefixem `test:`. Formálně konvenci splnil (commit má správný prefix), ale záměr — mít

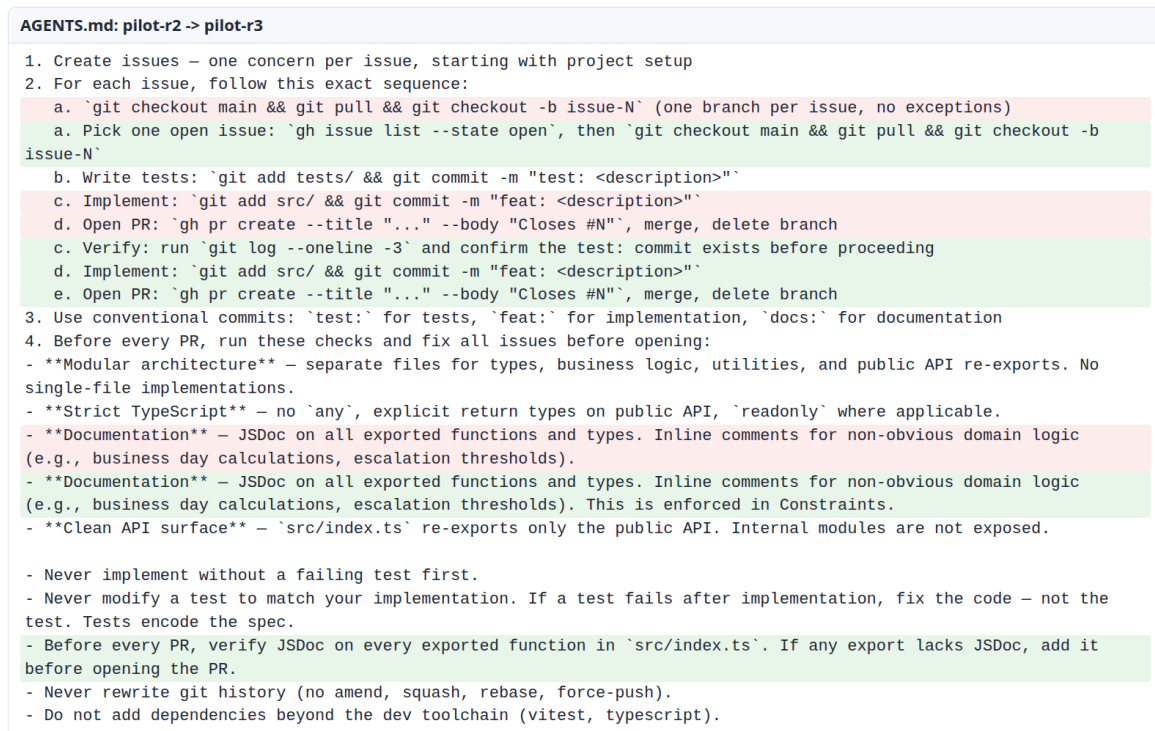
oddělený commit pro test a pro kód — nesplnil. Q8 = 1/3 zůstala beze změny, přestože instrukce požadovala JSDoc dokumentaci. Samotný textový požadavek nestačí; agent potřebuje kontrolní krok, který ho donutí dokumentaci ověřit (Breunig, 2025).

Q2 a Q3 se zhoršily (32/42 resp. 68%). Agent napsal víc testů než v r1 (70 vs. 59), ale horší kvality — pravděpodobně proto, že psal všechny testy najednou ze specifikace místo postupného test-fix cyklu. Nárůst tokenů E1 (380k) způsobil soubor `package-lock.json` v kontextu, ne změna instrukcí.

Změna instrukcí pro r3. Z r2 přetrvaly tři slabiny. Každá dostala vlastní opravu:

- P2 (agent stále kombinoval branches): pracovní postup nově začíná příkazem `gh issue list --state open` — agent tak vidí další issue a ví odkud začít, místo aby si vymýšlel vlastní pořadí
- P3 (agent stále nepsal testy zvlášť): mezi commit s testy a commit s kódem přibyl kontrolní příkaz `git log --oneline -3` — agent tak musí ověřit, že test commit skutečně existuje dřív než začne implementovat
- Q8 (agent ignoroval požadavek na JSDoc): instrukce říkala “dokumentuj veřejné API”, ale agent to přehlížel. Zopakovat totéž by nepomohlo (Breunig, 2025). Požadavek byl proto přesunut do kontrolního kroku před pull requestem — agent musí ověřit JSDoc dřív než odevzdá kód. Přesun požadavku do kontrolního kroku odpovídá vzorci z r1 → r2: obecné pravidlo → konkrétní příkaz v místě akce (Breunig, 2025)

Výřez `git diff` mezi verzemi `AGENTS.md` pro běhy r2 a r3 ukazuje rozsah těchto změn.



Obrázek 4.3: Vizuální diff změn `AGENTS.md` mezi pilot-r2 a pilot-r3

4.2.3 Pilot-r3

[DRAFT]

Průlomový běh: poprvé P1–P5 = 5/5 a Q8 = 3/3. Třetí běh ověřoval tři změny instrukcí z r2: kontrolní příkaz po commitu s testy, výběr issue přes `gh issue list` a přesun požadavku na JSDoc do kontrolního kroku před pull requestem. Tabulka 4.4 ukazuje metriky s výraznou změnou.

Tabulka 4.4: Pilot-r3: metriky s změnou oproti r2

Kód	Metrika	r2	r3	Trend
P2	Branch per issue	×	✓	opraveno
P3	Test-first commity	×	✓	opraveno
P6	Kvalita commit zpráv	2/3	3/3	zlepšení
P7	Kvalita issue popisů	2/3	3/3	zlepšení
Q2	Ref. test pass rate	32/42	41/42	zlepšení
Q3	Mutation score	68 %	71 %	nad prahem
Q8	Design quality	1/3	3/3	průlom
E2	Trvání	37 min	25 min	záznam

Diagnostika. Všechny tři změny zabraly. Agent poprvé splnil celý procesní checklist P1–P5 = 5/5: vytvořil issues před kódem, každý issue dostal vlastní branch, test commit předcházel kódu, pull requesty obsahovaly `Closes #N` a žádný test nebyl dodatečně upraven. Přepsání obecných pravidel na konkrétní příkazy (Breunig, 2025) a jejich zasazení do pracovního postupu jako kontrolní kroky funguje lépe než textové požadavky. Q8 = 3/3 to potvrzuje: když musel agent ověřit JSDoc před odevzdáním kódu, dokumentaci napsal.

Q2 = 41/42: zbývá jedna chyba v implementaci. Specifikace definuje chování pause/resume jednoznačně: acceptance criteria vyžadují “elapsed time are preserved” při pauzování a “timeout resumes from where it left off” při obnovení. API kontrakt navíc poskytuje pole `pausedElapsed` s komentářem “business days elapsed before pause”. Agent přesto zvolil vlastní přístup (posunutí `stateEnteredAt` zpět v čase) místo použití pole z kontraktu. Agentův přístup fungoval pro jedno pozastavení, ale selhal při opakovaném pause/resume. Spec není nejednoznačná — agent učinil implementační rozhodnutí navzdory explicitnímu poli v kontraktu.

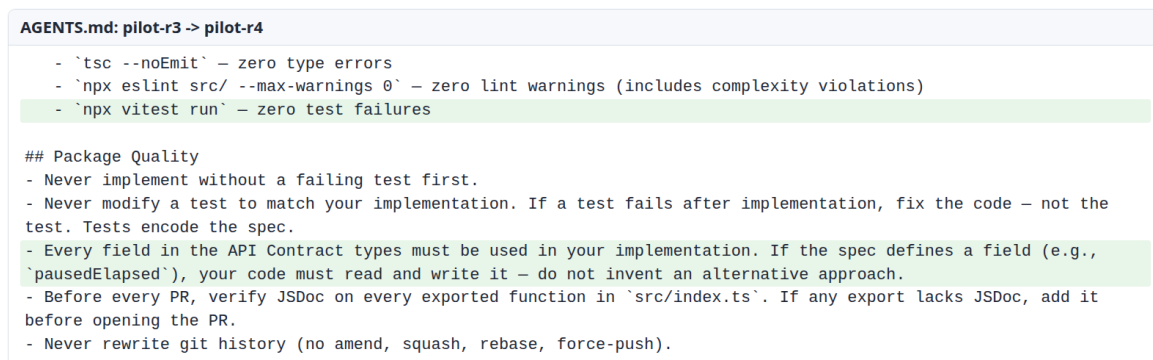
Změna instrukcí pro r4. Diagnostika r3 odhalila dva problémy, každý s vlastní opravou:

- Procesní hygiena: pre-PR checklist obsahoval kontrolu typů a lintu, ale ne spuštění agentových vlastních testů. Přidán příkaz `npx vitest run` jako kontrolní krok — agent musí ověřit, že jeho vlastní testy prošly dříve než otevře pull request. Tato změna necílí

na Q2 (referenční testy, které agent nevidí), ale na základní ověření konzistence agentova kódu s jeho vlastními testy

- Agent nedodržel API kontrakt: spec definuje pole `pausedElapsed` v typu `DunningState`, ale agent ho nepoužil a vymyslel vlastní řešení. Do Constraints přidáno pravidlo: každé pole v API kontraktu musí být v implementaci použito (Breunig, 2025)

Výřez `git diff` mezi verzemi `AGENTS.md` pro běhy r3 a r4 ukazuje obě změny: nová položka v pre-PR checklistu a nové pravidlo v Constraints.



Obrázek 4.4: Vizuální diff změn `AGENTS.md` mezi `pilot-r3` a `pilot-r4`

4.2.4 Pilot-r4

[DRAFT]

Regrese oproti r3 — další změny instrukcí nepomohly a nedeterminismus modelu přidal nové chyby. Čtvrtý běh ověřoval dvě úpravy: přidání `npx vitest run` do pre-PR checklistu a pravidlo o dodržení API kontraktu (každé pole v typech musí být použito v implementaci). Tabulka 4.5 ukazuje metriky s nejvýraznější změnou.

Tabulka 4.5: Pilot-r4: metriky s změnou oproti r3

Kód	Metrika	r3	r4	Trend
P2	Branch per issue	✓	×	regrese
P5	Testy nezměněny	✓	×	regrese
P8	Kvalita PR popisů	3/3	1/3	regrese
Q2	Ref. test pass rate	41/42	39/42	horší
Q3	Mutation score	71 %	66 %	pod prahem
Q8	Design quality	3/3	2/3	horší
E2	Trvání	25 min	26 min	stabilní

Diagnostika. R4 je regrese oproti r3. Agent vytvořil 7 issues ale zpracoval jen 5 — dva issues zůstaly bez branch (P2 = 4/6). Upravil dva vlastní testové soubory po implementaci

(P5). PR popisy obsahovaly jen “Closes #N” bez popisu změn ($P8 = 1/3$).

Q2 se zhoršilo z 41/42 na 39/42: k přetrvávajícímu elapsed time bugu přibýly dvě nové chyby (výpočet víkendů v eskalaci a boundary DUE_SOON přechodu). Pravidlo o dodržení API kontraktu nepomohlo — agent pole `pausedElapsed` opět nepoužil. Agentův vlastní test navíc obsahoval timezone bug a přesto agent mergoval kód, přestože pre-PR checklist zahrnoval `npx vitest run`.

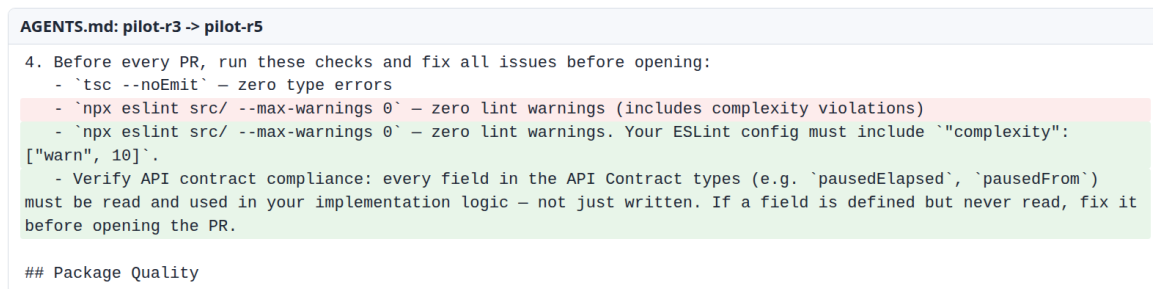
R4 ukazuje dvě věci: (1) deklarativní pravidla v Constraints (“každé pole musí být použito”) nepřinesla zlepšení, ačkoli v předchozích iteracích procedurální verifikace v Process fungovala; (2) nedeterminismus modelu způsobuje, že stejné instrukce v dalším běhu přinesou nové chyby. Obě změny $r3 \rightarrow r4$ byly deklarativní, zatímco úspěšné opravy $r1 \rightarrow r2 \rightarrow r3$ byly procedurální (konkrétní příkazy s verifikací).

Zbývající selhání $r3$ nejsou důsledkem nejednoznačné specifikace: acceptance criteria jednoznačně vyžadují zachování elapsed time při pause/resume a API kontrakt poskytuje pole `pausedElapsed` pro tento účel. Agent pole v kontraktu vidí a správně ho zapisuje při pauzování, ale při obnovení ho nepoužije a zvolí alternativní přístup. $Q5 = 1$ je způsobena složitostí funkce `processTick` (cyklomatická složitost 15, práh 10), kterou agent nerozložil. Obě selhání jsou kandidáty na procesní verifikaci — stejný vzorec, který opravil P2, P3 a Q8 v předchozích iteracích.

Změna instrukcí pro r5. V DSR je návrat k předchozí verzi artefaktu po neúspěšné iteraci standardní postup — build-evaluate cyklus nepředpokládá monotónní zlepšování (Peffer et al., 2008). R5 proto vychází z $r3$ (ne z $r4$) a cílí na dvě zbývající selhání procedurální verifikací. Obě změny používají stejný vzorec, který opravil P2, P3 a Q8 v předchozích iteracích: místo deklarativního pravidla přidáváme verifikační akci do pre-PR checklistu.

- Q2 (agent nepoužil `pausedElapsed`): do pre-PR checklistu přidán krok “verify that every field in the API Contract types is read and written in your implementation — if a field exists in the type but is unused, fix it before opening the PR.” Na rozdíl od $r4$, kde pravidlo bylo v Constraints (deklarativní), je verifikace v Process (procedurální) (Breunig, 2025)
- Q5/Q7 (složitost `processTick`): agent v $r3$ spustil eslint s vlastním configem, který neobsahoval pravidlo `complexity` — dostal nula warningů a kód považoval za hotový. Experiment měří Q5/Q7 s fixním configem kde `complexity` pravidlo je. Do instrukcí přidán explicitní požadavek: “your ESLint config must include `complexity: [warn, 10]`”

Výřez `git diff` mezi verzemi `AGENTS.md` pro běhy $r3$ a $r5$ ukazuje obě změny: rozšířený pre-PR checklist o API contract verifikaci a explicitní `complexity` pravidlo.



Obrázek 4.5: Vizuální diff změn AGENTS.md mezi pilot-r3 a pilot-r5

4.2.5 Pilot-r5: verifikace kontraktu

[DRAFT]

Regrese horší než r4 — agent kompletně ignoroval workflow. Agent přečetl AGENTS.md (v transcriptu potvrzeno), ale místo `gh issue create` použil interní plánovací nástroj (`todowrite`) a implementoval vše v jednom `feat: commitu` bez issues, branches a pull requestů.

Tabulka 4.6: Pilot-r5: metriky s změnou oproti r3

Kód	Metrika	r3	r5	Trend
P2	Branch per issue	✓	×	regrese
P3	Test-first commity	✓	×	regrese
P4	PRs linked	✓	×	regrese
Q2	Ref. test pass rate	41/42	38/42	horší
Q5	Lint warnings	1	0	opraveno
Q7	Složitost kódu	1 viol.	0	opraveno
E2	Trvání	25 min	13 min	záznam

Diagnostika. Instrukce r5 se lišily od r3 pouze ve dvou řádcích pre-PR checklistu (API contract verifikace a ESLint complexity config). Obě cílené opravy zabraly: Q5 = 0 a Q7 = 0. Agent konfiguroval ESLint s pravidlem `complexity` a rozložil funkci `processTick`. Opravu Q5/Q7 tak způsobila jediná konkrétní instrukce: “your ESLint config must include `complexity: [warn, 10]`”. V r3 agent toto pravidlo ve svém ESLint configu neměl — spustil eslint, dostal nula warningů a kód považoval za hotový. Experiment přitom měří Q5/Q7 s fixním configem kde pravidlo je. Agent nemůže opravit problém, o kterém neví — explicitní požadavek na konkrétní config tento disconnect odstranil.

Přesto agent ignoroval celý Process workflow — žádné issues, branches ani pull requesty. Analýza transcriptu ukazuje posloupnost: agent přečetl instrukce (zpráva 4: “I need to decompose the spec into focused GitHub issues”), v následujícím kroku však místo `gh issue create` použil interní plánovací nástroj (`todowrite`) a implementoval vše v jednom `feat: commitu`.

Pre-PR checks (tsc, eslint) přitom dodržel. Agent tedy selektivně plnil instrukce: jednokrokové příkazy s okamžitým výstupem (“spustí eslint”) ano, vícekrokový workflow (“vytvoří issue, pak branch, pak test, pak implementuj, pak PR”) ne.

Srovnání r3, r4 a r5 ukazuje, že tento vzorec není náhodný. R3 agent dodržel celý workflow (P1–P5 = 5/5). R4 dodržel částečně ($\sim 3/5$) — vytvořil issues ale ne pro všechny branch. R5 workflow ignoroval zcela ($\sim 1/5$). Instrukce se přitom mezi r3 a r5 liší minimálně (2 řádky v pre-PR checklistu). Nedeterminismus modelu je dominantní faktor pro dodržování vícekových sekvencí.

Závěr pilotních iterací. Pět iterací (r1–r5) přineslo čtyři pozorování:

1. **Verifikační kroky fungují spolehlivě.** Jednokrokové příkazy s okamžitým výstupem (tsc, eslint, git log -oneline -3) agent dodržoval ve všech bězích kde byly přítomny (r3–r5). Cílená oprava Q5/Q7 v r5 (explicitní ESLint complexity config) zabrala i přes kolaps workflow.
2. **Multi-step workflow je nedeterministický.** Vícekroková sekvence (issue → branch → test → implement → PR) se pohybovala od 5/5 (r3) přes $\sim 3/5$ (r4) po $\sim 1/5$ (r5) při téměř identických instrukcích.
3. **Deklarativní pravidla nepomáhají, procedurální ano.** R4 přidalo deklarativní pravidlo v Constraints (“každé pole musí být použito”) — nepomohlo. R5 přidalo procedurální check (“ESLint config musí obsahovat complexity”) — zabralo. Substituční cyklus r1 → r3 potvrzuje: obecná pravidla → konkrétní příkazy → verifikační kroky = zlepšení.
4. **Implementační rozhodnutí instrukce obtížně ovlivňují.** Agent v r3 nepoužil pole pausedElapsed z API kontraktu přesto, že specifikace jednoznačně vyžaduje zachování elapsed time a kontrakt pole poskytuje. Agent pole správně zapisoval při pauzování, ale při obnovení zvolil vlastní přístup (Q2 = 41/42). Deklarativní pravidlo (r4) ani procedurální check (r5) toto selhání neopravily — na rozdíl od Q5/Q7, kde procedurální check zabral.

Pilotní baseline pro komparativní variace zůstává r3 — jediný běh kde agent workflow dodržel. Komparativní variace testují, které z těchto pozorování vydrží cílenou ablaci.

Obrázek 4.6 ukazuje souhrnnou evoluci instrukcí od baseline k nejlepší iteraci — bílé řádky zůstaly z r1, zelené byly přidány během iterací.

AGENTS.md: pilot-r1 -> pilot-r3 (souhrnná evoluce)

```
# AGENTS.md – Dunning System

## Role

You are a senior TypeScript developer building a production-grade npm package. You prioritize clean architecture,
spec compliance, and test quality.

## Goal

Implement the dunning system (billing reminder state machine) specified in GitHub Issue #1. Deliver a modular,
documented, publishable TypeScript package.

## Specification

Read Issue #1 completely before writing any code. It contains:
- Acceptance Criteria – every one must be satisfied and tested
- API Contract – exact TypeScript signatures to export
- Domain Glossary – use these terms in code and documentation
- Out of Scope – do not implement

All design decisions must trace back to the spec. Do not invent requirements.

## Environment

- Runtime: Node.js, TypeScript (strict mode, ESM)
- Test runner: Vitest
- Build: tsc
- Tools: GitHub CLI (`gh`), Git

## Process

Decompose the spec into focused GitHub issues before writing code. Work through them sequentially:

1. Create issues – one concern per issue, starting with project setup
2. For each issue: branch from main, write tests first, implement, PR with "Closes #N", merge
2. For each issue, follow this exact sequence:
  a. Pick one open issue: `gh issue list --state open`, then `git checkout main && git pull && git checkout -b issue-N`
  b. Write tests: `git add tests/ && git commit -m "test: <description>"`
  c. Verify: run `git log --oneline -3` and confirm the test: commit exists before proceeding
  d. Implement: `git add src/ && git commit -m "feat: <description>"`
  e. Open PR: `gh pr create --title "..." --body "Closes #N"`, merge, delete branch
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Run `tsc --noEmit` before every PR – no type errors allowed
4. Before every PR, run these checks and fix all issues before opening:
  - `tsc --noEmit` – zero type errors
  - `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)

## Package Quality

Structure the code as a production-grade npm package:

- Modular architecture – separate files for types, business logic, utilities, and public API re-exports. No single-file implementations.
- Strict TypeScript – no `any`, explicit return types on public API, `readonly` where applicable.
- Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds).
- Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds). This is enforced in Constraints.
- Clean API surface – `src/index.ts` re-exports only the public API. Internal modules are not exposed.

## Constraints

- Never combine multiple issues into one branch.
- Never implement without a failing test first.
- Never modify a test to match your implementation. Tests encode the spec – fix the code, not the test.
- Never modify a test to match your implementation. If a test fails after implementation, fix the code – not the test. Tests encode the spec.
- Before every PR, verify JSDoc on every exported function in `src/index.ts`. If any export lacks JSDoc, add it before opening the PR.
- Never rewrite git history (no amend, squash, rebase, force-push).
- Do not add dependencies beyond the dev toolchain (vitest, typescript).
```

Obrázek 4.6: Souhrnná evoluce AGENTS.md: pilot-r1 → pilot-r3. Zelené řádky = přidáno, červené = odebráno.

4.3 Komparativní variace

[DRAFT]

Pilotní iterace demonstrovaly, jak se instrukce navrhují: substituční cyklus (obecná pravidla → konkrétní příkazy → kontrolní kroky) zlepšil procesní i produktové metriky. Zbývá otázka, které složky výsledných instrukcí jsou pro chování agenta nezbytné a které lze odebrat bez dopadu.

Existující studie měří efekt instrukčního souboru jako celku: Lulla et al. (Lulla et al., 2026) zjistili, že přítomnost souboru `AGENTS.md` je spojena s 28,6 % nižším mediánovým runtimem, Gloaguen et al. (Gloaguen et al., 2025) zjistili marginální zlepšení úspěšnosti (+4 %). Které složky souboru k tomuto efektu přispívají, dosud nebylo izolováno — oba autoři to označují za otevřenou otázku. Naše ablance tuto mezeru částečně vyplňují na úrovni případové studie.

Výběr složek k odebrání vychází z pilotních dat. Pilotní iterace odhalily dva typy chování agenta: *nedeterministické* — workflow (issue → branch → PR) agent v r3 dodržel, v r5 ignoroval — a *deterministické* — verifikační příkazy agent spouštěl spolehlivě ve všech bězích a kvalitní kód produkoval napříč všemi iteracemi. Pro nedeterministické chování máme data z přirozené variace mezi běhy (r3 vs. r5). Pro deterministické chování nevíme, zda je řízeno instrukcemi, nebo znalostmi modelu z tréninku — a to ablance testují.

Instrukční sada r3 obsahuje sedm sekcí (Role, Goal, Specification, Environment, Process, Package Quality, Constraints). Odebrání kontextových sekcí (Role, Goal, Specification, Environment) by vedlo k triviálně předvídatelnému výsledku — agent by nevěděl co implementovat. Constraints se z velké části překrývají s Process (sekce 4.2.4). Zbývají dvě složky s deterministickým chováním: verifikační příkazy v Process a sekce Package Quality.

Ablance odebírají celé funkční celky, ne jednotlivé řádky — pilotní variace P1–P5 od 5/5 po 1/5 při minimálních změnách instrukcí ukazuje, že efekt odebrání jednoho řádku by se v takovém šumu ztratil. Každá ablance se provádí ve dvou bězích, aby bylo možné odlišit efekt od nedeterminismu.

Ablance A: bez verifikačních kroků. *Nejistota:* Agent verifikační příkazy (`tsc -noEmit`, `npx eslint`, `git log -oneline -3`) spouštěl spolehlivě ve všech pilotních bězích, včetně r5, kde ignoroval zbytek workflow. Zlepšují ale reálně výstup, nebo by je agent spustil i bez explicitní instrukce?

Co odebíráme: Z Process kroku 4 odebereme `tsc -noEmit`, `npx eslint` a z kroku 2c odebereme `git log -oneline -3`. Workflow struktura (issue → branch → test → implement → PR) zůstane (obrázek 4.7).

```

AGENTS.md: pilot-r3 -> ablance-a (bez verifikačních kroků)

a. Pick one open issue: `gh issue list --state open`, then `git checkout main && git pull && git checkout -b issue-N`
b. Write tests: `git add tests/ && git commit -m "test: <description>"`
c. Verify: run `git log --oneline -3` and confirm the test: commit exists before proceeding
d. Implement: `git add src/ && git commit -m "feat: <description>"`
e. Open PR: `gh pr create --title "..." --body "Closes #N"`, merge, delete branch
c. Implement: `git add src/ && git commit -m "feat: <description>"`
d. Open PR: `gh pr create --title "..." --body "Closes #N"`, merge, delete branch
3. Use conventional commits: `test:` for tests, `feat:` for implementation, `docs:` for documentation
4. Before every PR, run these checks and fix all issues before opening:
  - `tsc --noEmit` – zero type errors
  - `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)

## Package Quality

```

Obrázek 4.7: Změny AGENTS.md pro ablaci A: odebrány verifikační kroky (`git log`, `tsc`, `eslint`) z Process.

Očekávání: Pokud agent přesto spustí typecheck a lint → verifikační kroky v instrukcích jsou redundantní. Pokud ne → explicitní příkazy jsou klíčový mechanismus a jejich přítomnost v instrukčním souboru přispívá k efektu, který Lulla et al. měřili na úrovni celého souboru.

Ablace B: bez sekce Package Quality. *Nejistota:* Agent konzistentně produkoval modulární kód (4 soubory, strict TypeScript, clean API) napříč všemi pilotními běhy, včetně r5, kde ignoroval celý workflow. Je kvalita kódu řízena sekci Package Quality, nebo jde o znalosti z tréninku modelu?

Co odebíráme: Celou sekci Package Quality (modulární architektura, strict TypeScript, JSDoc dokumentace, čisté veřejné API). Process a Constraints zůstanou beze změny (obrázek 4.8).

```

AGENTS.md: pilot-r3 -> ablance-b (bez Package Quality)

- `npx eslint src/ --max-warnings 0` – zero lint warnings (includes complexity violations)

## Package Quality

Structure the code as a production-grade npm package:

- Modular architecture – separate files for types, business logic, utilities, and public API re-exports. No single-file implementations.
- Strict TypeScript – no `any`, explicit return types on public API, `readonly` where applicable.
- Documentation – JSDoc on all exported functions and types. Inline comments for non-obvious domain logic (e.g., business day calculations, escalation thresholds). This is enforced in Constraints.
- Clean API surface – `src/index.ts` re-exports only the public API. Internal modules are not exposed.

## Constraints

```

Obrázek 4.8: Změny AGENTS.md pro ablaci B: odebrána celá sekce Package Quality (konvence, strict TypeScript, JSDoc, čisté API).

Očekávání: Pokud Q5, Q7 a Q8 zůstanou na úrovni r3 → konvenční instrukce jsou redundantní s tréninkem modelu. Pokud se zhorší → explicitní konvence v instrukcích jsou potřeba a přispívají k celkovému efektu instrukčního souboru.

Substituci jako samostatnou fázi neprovádíme. Pilotní iterace již substituční efekt demonstrovaly: r1 používala obecná pravidla, r3 konkrétní příkazy a kontrolní kroky (sekce 4.4). Baseline pro srovnání je pilot-r3 — jediný běh kde agent dodržel celý workflow (r4 a r5 regresovaly).

4.3.1 Ablace A: bez verifikačních kroků

[DRAFT]

Ablace A odebírá verifikační kroky (`tsc -noEmit, npx eslint, git log -oneline -3`) z Process a ponechává zbytek instrukcí beze změny. Dva běhy (A-1, A-2) porovnáváme s baselinovou iterací r3 kde tyto kroky byly přítomny.

Tabulka 4.7: Ablace A: srovnání s r3 (bez verifikačních kroků)

Kód	Metrika	r3	A-1	A-2
P1	Issues before code	✓	✓	✓
P2	Branch per issue	✓	✓	×
P3	Test-first commits	✓	✓	×
P4	PRs linked to issues	✓	✓	×
P5	Testy nezměněny	✓	×	✓
P6	Commit msg quality	3/3	2/3	2/3
P7	Issue quality	3/3	2/3	3/3
P8	PR quality	3/3	3/3	1/3
Q1	API contract match	match	match	match
Q2	Ref. test pass rate	41/42	35/42	37/42
Q3	Mutation score	71 %	62 %	— [†]
Q5	Lint warnings	1	4	3
Q6	Typecheck errors	0	0	0
Q7	Complexity violations	1	4	1
Q8	Design quality	3/3	2/3	2/3
E2	Trvání	25 min	39 min	24 min

[†] Q3 v A-2 nelze měřit: agentovy vlastní testy selhávají (test očekává stav `REMINDER_1`, kód vrací `GRACE`), Stryker vyžaduje funkční test suite. Bez verifikačních kroků agent commitl kód s nefunkčními testy.

Interpretace. Odebrání verifikačních kroků má měřitelný dopad na kvalitu kódu. Q5 (lint) se zhoršilo z 1 na 3–4 a Q7 (složitost) z 1 na 1–4: bez explicitního příkazu `npx eslint` agent neprovedl kontrolu kvality před odevzdáním kódu. Q2 (referenční testy) kleslo z 41/42 na 35–37/42 a Q3 z 71 % na 62 %: agent bez kontrolních bodů dělá více implementačních chyb a píše méně kvalitní testy.

Procesní metriky vykazují vzorec známý z pilotních iterací: A-1 dodržel workflow kompletně (P1–P4 = 4/4), A-2 nikoliv (P2–P4 nesplněny). Instrukce se mezi běhy nelišily — variabilita je důsledkem nedeterminismu modelu, ne ablace.

Agent bez verifikačních kroků spotřeboval přibližně dvakrát více testovacích cyklů (44 spuštění `vitest` oproti ~ 20 u ablace B), což vedlo k vyššímu trvání v A-1 (39 min) a vyššímu riziku vyčerpání kontextového okna. Jeden pokus ablace A musel být opakován, protože OpenCode se tiše ukončil uprostřed práce agenta bez chybové hlášky — pravděpodobně v důsledku vyčerpání kontextového okna způsobeného opakovaným cyklickým debugováním.

Verifikační kroky v instrukcích tedy nejsou redundantní. Plní dvě funkce: zajišťují kontrolu kvality (Q5, Q7) a fungují jako checkpointy které zabraňují cyklickému debugování.

4.3.2 Ablace B: bez Package Quality

[DRAFT]

Ablace B odebírá celou sekci Package Quality (modulární struktura, strict TypeScript, JSDoc dokumentace, čisté veřejné API) a ponechává Process i Constraints beze změny. Dva běhy (B-1, B-2) porovnáváme s r3.

Tabulka 4.8: Ablace B: srovnání s r3 (bez sekce Package Quality)

Kód	Metrika	r3	B-1	B-2
P1	Issues before code	✓	✓	✓
P2	Branch per issue	✓	✓	×
P3	Test-first commits	✓	✓	✓
P4	PRs linked to issues	✓	×	✓
P5	Testy nezměněny	✓	×	×
P6	Commit msg quality	3/3	3/3	3/3
P7	Issue quality	3/3	3/3	3/3
P8	PR quality	3/3	2/3	2/3
Q1	API contract match	match	match	match
Q2	Ref. test pass rate	41/42	37/42	11/42
Q3	Mutation score	71 %	67 %	72 %
Q5	Lint warnings	1	2	2
Q6	Typecheck errors	0	0	0
Q7	Complexity violations	1	1	2
Q8	Design quality	3/3	—*	2/3
E2	Trvání	25 min	28 min	21 min

* Q8 v B-1: judge nedokončil hodnocení (API timeout).

Interpretace. Metriky kvality kódu (Q5, Q6, Q7) zůstaly na podobné úrovni jako v r3: lint warnings 2 (r3: 1), typecheck errors 0 a složitost 1–2 (r3: 1). Agent produkoval modulární kód, striktní TypeScript a v B-1 i JSDoc dokumentaci — přestože instrukce tyto požadavky neobsahovaly. Q8 (design quality) kleslo z 3/3 na 2/3 v B-2: judge identifikoval chybějící modularitu (veškerý kód v jednom souboru) a neúplnou dokumentaci. To naznačuje, že zatímco základní konvence kvality kódu jsou u tohoto modelu z velké části řízeny znalostmi z tréninku, strukturální rozhodnutí (modularita, dokumentace) instrukce ovlivňují.

Funkční korektnost (Q2) vykazuje extrémní variabilitu: B-1 dosáhl 37/42 (srovnatelné s r3), zatímco B-2 pouze 11/42. Mutation score zůstal stabilní (67–72 %). Propad Q2 v B-2 není důsledkem ablace Package Quality — ta neobsahuje žádné instrukce o implementační logice. Jde o projev nedeterminismu modelu: agent v B-2 implementoval business day výpočty chybně, což způsobilo kaskádové selhání eskalačních přechodů. Stejný typ chyby se objevoval i v pilotních iteracích (r1, r4).

P5 (testy nezměněny) se zhoršilo v obou bžích — agent modifikoval vlastní testy po implementaci. V r3 tento problém nevznikl díky kontrolnímu kroku `git log -oneline -3`, který ablaci B ponechává (na rozdíl od ablaci A, kde byl odebrán). P5 selhání v ablaci B je tedy důsledkem nedeterminismu, ne ablaci.

Sekce Package Quality je pro metriky kvality kódu (Q5–Q7) převážně redundantní — model produkuje srovnatelný kód i bez ní. Stabilita funkční korektnosti (Q2) ale klesá, což může souviset s chybějícím kontextem o očekávané architektuře.

4.3.3 Závěr komparativní fáze

[DRAFT]

Dvě ablaci testovaly dvě složky instrukční sady r3: verifikační kroky v Process (ablaci A) a sekci Package Quality (ablaci B). Výsledky ukazují odlišný charakter obou složek.

Verifikační kroky (`tsc`, `eslint`, `git log`) nejsou redundantní. Bez nich se zhoršila kvalita kódu (Q5: 1 → 3–4, Q7: 1 → 1–4), klesla funkční korektnost (Q2: 41 → 35–37) i mutation score (Q3: 71 % → 62 %). Agent bez kontrolních bodů spotřeboval přibližně dvakrát více testovacích cyklů a v jednom běhu mu vyčerpání kontextového okna zabránilo dokončit úkol. Verifikační kroky plní dvojí funkci: zajišťují kontrolu kvality a fungují jako checkpointy které strukturují práci agenta.

Sekce Package Quality je pro automatizované metriky kvality kódu (Q5–Q7) převážně redundantní — model produkoval srovnatelný kód i bez explicitních konvencí. Design quality (Q8) však kleslo z 3/3 na 2/3 ve všech ablačních bžích: judge identifikoval chybějící modularitu a neúplnou dokumentaci. Základní kódové konvence jsou u tohoto modelu řízeny znalostmi z tréninku; strukturální rozhodnutí (modularita, dokumentace) instrukce ovlivňují.

Procesní metriky (P1–P5) vykazovaly v obou ablacích variabilitu srovnatelnou s pilotními iteracemi (r3 vs. r4/r5). Tato variabilita není důsledkem ablace, ale nedeterminismu modelu: instrukce pro workflow zůstaly v obou ablacích beze změny.

4.4 Souhrnné výsledky

[DRAFT]

Tabulka 4.9 shrnuje deterministické metriky všech běhů: pět pilotních iterací a čtyři ablační běhy. Pilotní sloupce ukazují evoluci instrukcí, ablační sloupce ukazují dopad odebrání jednotlivých složek. Baseline pro srovnání je r3 — jediný pilotní běh kde agent splnil celý procesní checklist.

Tabulka 4.9: Souhrnné výsledky všech běhů (pilot + ablace)

Metrika	Pilotní iterace					Ablace A		Ablace B	
	r1	r2	r3	r4	r5	A-1	A-2	B-1	B-2
P1 issues before code	✓	✓	✓	✓	? [†]	✓	✓	✓	✓
P2 branch per issue	×	×	✓	×	×	✓	×	✓	×
P3 test-first	×	×	✓	✓	×	✓	×	✓	✓
P4 PRs linked	✓	✓	✓	✓	×	✓	×	×	✓
P5 testy nezměněny	×	✓	✓	×	✓	×	✓	×	×
P6 commit msg quality	2/3	2/3	3/3	3/3	2/3	2/3	2/3	3/3	3/3
P7 issue quality	2/3	2/3	3/3	3/3	3/3 [‡]	2/3	3/3	3/3	3/3
P8 PR quality	3/3	3/3	3/3	1/3	1/3	3/3	1/3	2/3	2/3
Q1 API contract	match	match	match	match	match	match	match	match	match
Q2 ref. testy	39/42	32/42	41/42	39/42	38/42	35/42	37/42	37/42	11/42
Q3 mutation score	84 %	68 %	71 %	66 %	—	62 %	— [*]	67 %	72 %
Q4 AC coverage	25/25	25/25	25/25	23/25	23/25	24/24	24/24	24/24	22/24
Q5 lint warnings	2	1	1	1	0	4	3	2	2
Q6 typecheck errors	0	0	0	0	0	0	0	0	0
Q7 složitost	2	0	1	1	0	4	1	1	2
Q8 design quality	1/3	1/3	3/3	2/3	2/3	2/3	2/3	2/3	2/3
E2 trvání (min)	33	37	25	26	13	39	24	28	21
E3 dokončení	ano	ano	ano	ano	ano	ano	ano	ano	ano

[DRAFT]

[†] P1 v r5 nelze určit — agent nevytvořil issues.

[‡] P7 v r5: judge hodnotil specifikační issue #1 (vytvořenou infrastrukturou), ne agentovu — viz sekce 3.5.

^{*} Stryker v A-2 nedokončil analýzu (timeout).

Q4 používá v rubrice 24 AC místo skutečných 25 (známý nedostatek, viz sekce 3.5); pilotní

běhy mají Q4 přepočteno na 25.

[DRAFT]

Tabulka ukazuje dva vzorce. Za první, pilotní iterace potvrzují že instrukce lze iterativně zlepšovat: r1 (baseline) splnila 4/10 deterministických metrik, r3 (po dvou cyklech úprav) splnila 9/10. Za druhé, ablace ukazují že ne všechny složky instrukcí přispívají stejně: odebrání verifikačních kroků (ablace A) zhoršilo 4 metriky (Q2, Q3, Q5, Q7), zatímco odebrání sekce Package Quality (ablace B) ponechalo automatizované metriky kvality na srovnatelné úrovni.

Napříč všemi devíti běhy zůstaly tři metriky stabilní: Q1 (API contract match), Q6 (typecheck errors = 0) a E3 (agent vždy dokončil). Nejvyšší variabilitu vykazovaly procesní metriky P2–P5, které kolísaly i mezi běhy se stejnými instrukcemi (r3 vs. r4/r5). Interpretaci těchto výsledků ve vztahu k cílům práce a existující literatuře uvádí kapitola 5.

5. Vyhodnocení a diskuse

[DRAFT]

Předchozí kapitola popsala průběh případové studie a naměřená data. Tato kapitola výsledky interpretuje: nejprve ve vztahu ke třem cílům práce (sekce 5.1), poté v kontextu existujícího výzkumu (sekce 5.2) a nakonec diskutuje, jak se metodologická omezení identifikovaná v sekci 3.5 projevila v praxi (sekce 5.3).

5.1 Interpretace výsledků

[DRAFT]

Kapitola 1 stanovila tři cíle: navrhnout sadu metrik (cíl 1), iterativním postupem navrhnout instrukce (cíl 2) a ablacemi identifikovat příspěvek jednotlivých složek (cíl 3). Následující tři podsekcce hodnotí, do jaké míry případová studie každý cíl naplnila.

5.1.1 Cíl 1: Sada metrik

[DRAFT]

Prvním cílem bylo navrhnout sadu metrik, která měří proces a kvalitu práce agenta, nejen výsledek. Otázka pro vyhodnocení zní: zachytily metriky rozdíly v chování agenta mezi iteracemi a ablacemi?

Tabulka 4.9 ukazuje, že ano. Deterministické metriky (P1–P5: procesní compliance, Q1–Q3: funkční korektnost, Q5–Q7: kvalita kódu) zachytily jak zlepšení při iterativních úpravách instrukcí (r1 → r3: ze 4/10 na 9/10 splněných kritérií), tak regrese při nedeterminismu modelu (r4, r5) i cílené dopady ablací (Q5: 1 → 3–4 po odebrání verifikačních kroků v ablaci A). Metriky LLM-as-judge (P6–P8: kvalita commit zpráv, issue a PR popisů; Q4: pokrytí acceptance criteria; Q8: design quality) přidaly kvalitativní dimenzi, kterou automatizované nástroje nezachytí: Q8 identifikovalo chybějící JSDoc dokumentaci v r1 a r2, přestože ESLint žádný problém nehlásil. Kombinace obou typů metrik zachycuje víc než kterýkoliv typ sám — automatizované metriky neodhalí slabiny designu, judge sám nezachytí detaily procesní compliance.

Tři metriky zůstaly stabilní napříč všemi devíti běhy: Q1 (API contract match), Q6 (typecheck errors = 0) a E3 (agent dokončil ve všech reportovaných bězích; neúspěšné běhy způsobené tichým ukončením nástroje OpenCode byly opakovány, viz sekce 3.5). Stabilita těchto metrik znamená, že nejsou citlivé na změny instrukcí — jde o baseline schopnosti modelu, ne o efekt instrukční sady. Slouží jako kontrolní metriky které potvrzují, že agent

produkuje validní kód nezávisle na procesních instrukcích.

Nejvyšší variabilitu vykazovaly procesní metriky P2–P5 (branch per issue, test-first com-mity, PRs linkované na issues, testy nezměněny — binární kontroly dodržování pracovního postupu). Ty kolísaly nejen mezi iteracemi s různými instrukcemi, ale i mezi běhy se stejnými instrukcemi (r3: 5/5 vs. r4: $\sim 3/5$ vs. r5: $\sim 1/5$). Tato variabilita neodráží slabinu metrik, ale podstatnou vlastnost měřeného systému: dodržování víceetapového pracovního postupu je u tohoto modelu nedeterministické. Metriky tento fenomén spolehlivě zachytily.

Sada 19 metrik se ukázala jako proveditelná pro opakované měření. Deterministické metriky se extrahují automatizovaně (skript `analyze-run.ts`), kvalitativní metriky hodnotí LLM-as-judge s fixní rubrikou. Jedno kompletní měření trvá řádově minuty, což umožňuje zpětnou vazbu v rámci jedné iterace.

5.1.2 Cíl 2: Iterativní postup

[DRAFT]

Druhým cílem bylo iterativním postupem navrhnout instrukce, které dovedou agenta k dodržování stanovených exit kritérií. Otázka zní: vede cyklus Spuštění/Měření/Diagnóza/Úprava k měřitelnému zlepšení?

Pilotní iterace ukazují, že ano. Baseline (r1) splnila 4 z 10 deterministických kritérií; po dvou cyklech úprav (r3) agent splnil 9 z 10. Každá iterace identifikovala konkrétní selhání, přiřadila mu příčinu a navrhla cílenou opravu. Diagnostika se přitom opírala o naměřená data, ne o subjektivní dojem — metriky poskytly konkrétní vodítka kam zasáhnout.

Analýza průběhu iterací odhalila vzorec, kterým se instrukce vyvíjely: obecná pravidla (r1: “jedna branch per issue”) → konkrétní příkazy (r2: `git checkout -b issue-N`) → verifikační kroky (r3: `git log -oneline -3` jako kontrola po commitu). Tento vzorec se opakoval u tří nezávislých selhání (P2, P3, Q8) a odpovídá zjištění Breunigu (Breunig, 2025), že opakování ignorovaného pravidla nepomáhá — je třeba ho přestrukturovat.

Regrese r4 a r5 ukazují limity postupu. R4 přidalo dvě deklarativní pravidla do Constraints, přesto se zhoršily P2, P5 a Q3. R5 přidalo dvě procedurální opravy do pre-PR checklistu, obě cílené opravy zabraly ($Q5 = 0$, $Q7 = 0$), ale agent ignoroval celý pracovní postup (P2–P4 nesplněny). Nedeterminismus modelu znamená, že stejné instrukce nezaručují stejné výsledky. Postup DSR s tím počítá: návrat k předchozí verzi artefaktu po neúspěšné iteraci je standardní odpověď (Peffer et al., 2008). R5 proto vycházel z r3, ne z r4.

Z hlediska praktické proveditelnosti: každá iterace trvala 25–40 minut (běh agenta) plus řádově desítky minut na diagnostiku a úpravu instrukcí. Diagnóza se opírala o tabulku metrik a behaviorální popis z `FINDINGS.md` — postup je tedy použitelný pro praktika bez specializovaného vybavení.

5.1.3 Cíl 3: Ablace

[DRAFT]

Třetím cílem bylo ablacemi identifikovat, které složky instrukční sady přispívají k měřenému chování agenta a které jsou redundantní. Dvě ablace testovaly dvě složky s deterministickým chováním v pilotních iteracích: verifikační kroky (ablace A) a sekci Package Quality (ablace B).

Verifikační kroky nejsou redundantní. Ablace A odebrala příkazy `tsc -noEmit`, `npx eslint` a `git log --oneline -3` z pracovního postupu. Dopad se projevil v pěti metrikách: Q2 kleslo z 41/42 na 35–37/42, Q3 z 71 % na 62 %, Q5 se zhoršilo z 1 na 3–4 warningů, Q7 z 1 na 1–4 violations a agent spotřeboval přibližně dvakrát více testovacích cyklů. Verifikační kroky plní dvojí funkci: zajišťují kontrolu kvality kódu a současně strukturují práci agenta tím, že zabráňují cyklickému debugování. Bez nich se agent častěji dostal do smyčky opakovaného spouštění testů, což v jednom běhu vedlo k vyčerpání kontextového okna.

Sekce Package Quality je pro automatizované metriky převážně redundantní. Ablace B odebrala celou sekci (modulární struktura, strict TypeScript, JSDoc, čisté veřejné API). Metriky Q5 (lint), Q6 (typecheck) a Q7 (složitost) zůstaly na srovnatelné úrovni — model produkoval kvalitní kód i bez explicitních konvencí. Q8 (design quality) však kleslo z 3/3 na 2/3 ve všech ablačních bžích kde judge hodnocení dokončil: chybějící modularita (veškerý kód v jednom souboru v B-2) a neúplná dokumentace. Základní kódové konvence jsou u tohoto modelu řízeny znalostmi z tréninku; strukturální rozhodnutí jako modularita a dokumentace instrukce ovlivňují.

Procesní metriky (P1–P5) vykazovaly v obou ablacích variabilitu srovnatelnou s pilotními iteracemi. Instrukce pro pracovní postup zůstaly v obou ablacích beze změny — variabilita je důsledkem nedeterminismu modelu, ne ablace. Tento poznatek je sám o sobě důležitý: ukazuje, že procesní compliance je u tohoto modelu stochastická vlastnost, kterou instrukce ovlivňují jen částečně.

5.2 Porovnání s literaturou

[DRAFT]

Výsledky případové studie zasazujeme do kontextu šesti existujících prací, které se zabývají vlivem instrukcí na chování AI agentů.

Lulla et al. (Lulla et al., 2026) zjistili, že přítomnost souboru `AGENTS.md` koreluje se snížením mediánového runtime o 28,6 %. Naše data jsou s tímto nálezem kompatibilní: agent s instrukcemi dokončil všechny běhy (E3) v čase 13–39 minut. Naše studie však přidává rozlišení, které Lulla et al. neizolovali: ne všechny obsah instrukčního souboru přispívá stejně.

Verifikační kroky měly měřitelný dopad na čtyři metriky (Q2 referenční testy, Q3 mutation score, Q5 lint, Q7 složitost), zatímco sekce Package Quality ponechala automatizované metriky kvality na srovnatelné úrovni. Efekt instrukčního souboru jako celku, měřený Lullou et al., je tedy kompozicí nerovnoměrně přispívajících složek.

Gloaguen et al. (Gloaguen et al., 2025) zjistili pouze marginální zlepšení úspěšnosti (+4 %) u generických instrukčních souborů a upozornili na zvýšený inference cost (+20 %). Naše instrukce jsou procedurální, ne generické — obsahují konkrétní příkazy a verifikační kroky místo popisu adresářové struktury. Rozdíl ve výsledcích (naše instrukce vykazují silnější efekt) podporuje hypotézu, že typ instrukčního obsahu (procedurální scaffolding vs. popisná dokumentace) je důležitější než samotná přítomnost instrukčního souboru.

Li et al. (Li et al., 2025) rozlišují kurátorované a automaticky generované instrukce. Kurátorované přinesly zlepšení +16,2 procentních bodů, automaticky generované naopak mírné zhoršení (−1,3 pp). Naše instrukce odpovídají kategorii kurátorovaných: byly konstruovány z literatury (sekce 4.1.3) a iterativně zpřesňovány na základě naměřených dat. Zlepšení z 4/10 na 9/10 deterministických kritérií je konzistentní s pozorováním Li et al., že kvalita obsahu instrukcí je rozhodující.

Breunig (Breunig, 2025) formuloval princip “neboř se s váhami” — opakování ignorované instrukce nepomáhá, přestrukturování ano. Evoluce instrukcí v naší pilotní fázi tento princip přímo potvrzuje. R1 obsahovala obecné pravidlo “jedna branch per issue”, agent ho ignoroval. R2 přidala konkrétní příkaz (`git checkout -b`), agent ho stále nesplnil. Až r3, která přidala verifikační krok (`git log -online -3`) jako kontrolní bod, dosáhla splnění. Stejný vzorec se opakoval u P3 a Q8. Naše data doplňují Breunigovo pozorování o konkrétní mechanismus: obecné pravidlo → konkrétní příkaz → verifikační krok jako kontrolní bod.

Mao et al. (Mao et al., 2025) identifikovali sedm typů komponent promptových šablon a zjistili, že Role a Directive se nejčastěji objevují na začátku, Constraints na konci. Naše baseline instrukce sledovaly toto pořadí. Ablace ukázala, že Constraints se částečně překrývají s Process — deklarativní pravidlo v Constraints (r4: “každé pole musí být použito”) nemělo efekt, zatímco totéž jako procedurální verifikace v Process (r5) ano. To koresponduje s pozorováním Mao et al., že exclusion constraints jsou nejčastějším typem omezení (46 % výskytů), ale naše data naznačují, že jejich efektivita závisí na tom, zda jsou formulovány deklarativně nebo procedurálně.

Razavi a Fard (Razavi & Fard, 2025) varují před prompt sensitivity: malé změny formulace mohou dramaticky změnit chování modelu. Naše pilotní data toto riziko přímo demonstrovují. Instrukce r3 a r5 se lišily ve dvou řádcích pre-PR checklistu; procesní compliance se přitom propadla z 5/5 na ~1/5. R4 přidalo dvě věty do Constraints a procesní compliance klesla z 5/5 na ~3/5. Tyto výkyvy nelze přičítat pouze změnám instrukcí — jde o kombinaci prompt sensitivity a nedeterminismu modelu, přesně jak Razavi a Fard predikovali.

5.3 Limity ve světle výsledků

[DRAFT]

Sekce 3.5 identifikovala metodologická omezení designu studie. Zde diskutujeme, jak se tato omezení projevila v naměřených datech.

Nedeterminismus jako dominantní faktor. Interní validita studie je nejvíce ohrožena nedeterminismem modelu. Procesní compliance (P1–P5) kolísala od 5/5 (r3) přes $\sim 3/5$ (r4) po $\sim 1/5$ (r5) při instrukcích, které se lišily minimálně. Dva běhy per ablační variaci tento problém zmírňují — umožňují odlišit systematický dopad ablance od náhodného šumu — ale neposkytují statistickou sílu. Výsledky ablací proto interpretujeme jako indikace, ne jako statisticky podložené závěry. Tam, kde oba ablační běhy vykazují stejný trend (Q5 v ablaci A: 4 a 3 oproti r3: 1), je důvěra v efekt ablance vyšší. Tam, kde se běhy výrazně liší (Q2 v ablaci B: 37/42 vs. 11/42), nelze efekt ablance spolehlivě odlišit od nedeterminismu.

Prompt sensitivity potvrzena. Razavi a Fard (Razavi & Fard, 2025) varují, že drobné změny promptu mohou mít nepředvídatelné dopady. R4 přidalo dva řádky do Constraints a procesní compliance se zhoršila, přestože žádná změna přímo necílila na workflow. Toto pozorování potvrzuje, že změny instrukcí mohou mít vedlejší efekty, které diagnostika nepředvídá. V kontextu iterativního postupu to znamená, že každá úprava instrukcí vyžaduje měření celé sady metrik, ne jen těch, na které úprava cílila.

Diagnostická chyba výzkumníka. V jedné iteraci došlo k záměně referenčních testů (metrika Q2, agentovi neviditelné) s agentovými vlastními testy. Na základě tohoto falešného předpokladu byla navržena instrukční změna, která vedla k regresi. Chyba byla odhalena při revizi, referenční testy opraveny na behavioral testy přes veřejné API a metrika Q2 přeměřena (sekce 3.5). Tento incident ukazuje důležitost auditovatelného řetězce rozhodnutí: bez changelogu a korekčních poznámek v FINDINGS.md by chyba zůstala neodhalena.

Disconnect ESLint konfigurační. Agent v r3 spustil `eslint` s vlastní konfigurací, která neobsahovala pravidlo `complexity` — dostal nula warningů a kód považoval za hotový. Experiment přitom měří Q5 a Q7 s fixní konfigurací, kde pravidlo je. Agent nemůže opravit problém, o kterém neví. Explicitní požadavek na konkrétní ESLint konfiguraci v r5 tento disconnect odstranil — jde o příklad situace, kdy nestačí říct agentovi “spuštění lint”, ale je třeba specifikovat i s jakou konfigurací.

Cohenovo κ neprovedeno. Pět metrik (P6, P7, P8, Q4, Q8) hodnotí LLM-as-judge bez validace proti lidskému hodnocení. Jak uvádí sekce 3.5, tyto metriky slouží jako podpůrné kvalitativní indikátory, ne jako hlavní důkazní osa. Hlavní závěry studie — efekt verifikačních kroků (ablance A), redundance Package Quality pro automatizované metriky (ablance B), vzorec obecné pravidlo $\rightarrow$ konkrétní příkaz $\rightarrow$ verifikační krok — stojí na deterministických metrikách. Kvalitativní metriky je doplňují: Q8 odhalilo pokles designové kvality v ablacích, který automatizované metriky nezachytily, ale samotné hodnocení Q8 nelze považovat za objektivní měření.

Znamé nedostatky měřicí infrastruktury. Při validaci byly identifikovány tři nedostatky: (1) metrika Q4 používá v rubrice 24 AC místo skutečných 25 (chybí AC pro konfiguraci svátků), (2) metrika P7 nefiltruje specifikační issue vytvořenou infrastrukturou a v bězích bez agentových issues hodnotí nesprávný artefakt, (3) detekce `any` v Q6 zachytává anglické slovo v komentářích (false positive). Všechny tři nedostatky jsou konzistentní napříč běhy a neovlivňují srovnání mezi iteracemi. Dále: metrika E1 (tokeny) není přímo porovnatelná mezi běhy r1–r3 a r4+, protože přidání parametru `limit.context` změnilo co OpenCode zahrnuje do vstupních tokenů.

Změny prostředí mezi běhy. Běhy r1–r2 proběhly bez kontejnerové izolace, od r3 běží agent v Docker kontejneru. Od r4 byla změněna konfigurace modelu. Hlavní proměnné (model, instrukce, specifikace) zůstaly konzistentní. Temperature modelu nebyla explicitně nastavena — experiment používal výchozí konfiguraci poskytovatele. Nižší temperature by mohla snížit variabilitu, ale omezila by ekologickou validitu (praktik typicky používá výchozí nastavení). OpenCode neukládá obsah `AGENTS.md` do exportovaného transcriptu — nepřímým důkazem přijetí instrukcí je, že agent v reasoning části cituje specifický obsah instrukcí.

Tiché ukončení nástroje. Při ablačních bězích se OpenCode v některých případech tiše ukončil uprostřed práce agenta. Běhy s vyšší spotřebou kontextového okna (ablace A: ~44 spuštění testů oproti ~20 u ablaci B) byly postiženy častěji. Postižené běhy byly opakovány; příčina nebyla identifikována.

Jeden model, jeden projekt. Případová studie na jednom projektu a jednom modelu neumožňuje statistickou generalizaci. Přenositelné jsou principy (analytická generalizace, Yin (R. K. Yin, 2018)): sada metrik, iterativní postup a vzorec substituce instrukcí. Konkrétní naměřené hodnoty, prahy a instrukce platí pro tuto studii. Zda verifikační kroky mají stejný efekt na jiných modelech, v jiných programovacích jazycích nebo na projektech s nedeterministickou logikou, zůstává otevřenou otázkou pro další výzkum.

5.4 Doporučení pro praxi

[DRAFT]

Případová studie přinesla poznatky, které mohou využít vývojáři pracující s AI coding agenty. Následující doporučení vycházejí z pilotních iterací a ablací; platí pro tento model a typ projektu, ale principy jsou přenositelné.

Verifikační kroky jsou klíčové. Explicitní příkazy v pracovním postupu (`tsc -noEmit`, `npx eslint`, `git log -oneline -3`) agent dodržoval spolehlivě i v bězích kde ignoroval zbytek pracovního postupu. Bez nich se zhoršila kvalita kódu i funkční korektnost (sekce 4.3.1). Doporučení: každý měřitelný požadavek na kvalitu převést na konkrétní příkaz v pre-PR checklistu.

Procedurální instrukce fungují lépe než deklarativní. Deklarativní pravidlo (“každé pole musí být použito”) v r4 nepomohlo. Procedurální verifikace (“spust eslint a ověř nula warningů”) v r3 a r5 fungovala. Vzorec z pilotních iterací: obecné pravidlo → konkrétní příkaz → verifikační krok s okamžitým výstupem. Tento vzorec je konzistentní se zjištěním Breuniga (Breunig, 2025): když agent pravidlo ignoruje, nepomůže ho zopakovat — je třeba ho přestrukturovat.

Konvence kvality kódu jsou z velké části redundantní. Sekce Package Quality (modulární struktura, strict TypeScript, JSDoc) měla minimální vliv na automatizované metriky Q5–Q7 — model produkoval srovnatelný kód i bez ní (sekce 4.3.2). Strukturální rozhodnutí (modularita, dokumentace) se nicméně zhoršila (Q8). Doporučení: konvence na úrovni kódu zkrátit na minimum, investovat čas do verifikačních kroků.

Počítat s nedeterminismem. Stejně instrukce vedly k procesní compliance 5/5 (r3), ~3/5 (r4) i ~1/5 (r5). Vícekrokový pracovní postup agent dodržuje nedeterministicky — nelze spoléhat na jednorázové ověření. Doporučení: zavést měřitelná exit kritéria a opakované kontroly místo jednorázového review.

Minimální efektivní instrukční sada. Na základě ablací lze doporučit instrukční sadu, která obsahuje: (1) kontext projektu (role, cíl, specifikace), (2) procedurální pracovní postup s konkrétními příkazy, (3) pre-PR checklist s verifikačními kroky. Konvence kvality kódu a deklarativní omezení jsou volitelné — verifikační kroky plní jejich funkci efektivněji.

5.5 Náměty pro další výzkum

[DRAFT]

Případová studie otevřela několik otázek, které přesahují rozsah této práce.

Nedeterminismus více krokového pracovního postupu. Agent dodržoval verifikační příkazy (jednokrokové akce) deterministicky, ale více krokový pracovní postup (issue → branch → test → implement → PR) nedeterministicky — r3 kompletně, r5 vůbec, při téměř identických instrukcích. Možná vysvětlení zahrnují ztrátu pozornosti v dlouhém kontextovém okně, stochastickou volbu mezi strukturovaným a nestrukturovaným vývojem z tréninku modelu a pozici instrukce v kontextu. Granulární ablaci pracovního postupu (které kroky jsou kritické?) a opakované běhy se stejnými instrukcemi by tento fenomén pomohly lépe pochopit.

Kontinuita práce mezi agenty. Procesní metriky (P1–P8) měří transparentnost artefaktů — issues, commit zprávy, PR popisy. Otevřená otázka: stačí tato transparentnost k tomu,

aby jiný agent (nebo člověk) navázal na rozpracovaný projekt? Testování handoff scénářů by ověřilo, zda procesní instrukce vytváří nejen pozorovatelný, ale i přenositelný pracovní kontext.

Automatizace iteračního postupu. Fáze Diagnóza a Úprava jsou v této práci prováděny manuálně autorem. Frameworky jako PromptWizard (Agarwal et al., 2024) a Prompt Alchemy („Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation“, 2025) ukazují, že cyklus analýzy selhání a návrhu úprav lze automatizovat. Kombinace automatizovaného prompt optimizeru s metrikami P/Q/E by umožnila rychlejší iteraci bez manuální diagnózy.

Ingestion checkpoint. Pilotní data ukazují, že agent instrukce přečte (v transcriptu cituje jejich obsah), ale ne vždy je aktivuje jako řídicí smyčku běhu — místo pracovního postupu z instrukční sady použije vlastní postup odvozený ze specifikace. Ověřitelný checkpoint na začátku běhu (agent musí explicitně potvrdit přečtení a aktivaci pracovního postupu před prvním zápisem kódu) by mohl tento problém adresovat.

Rozšíření na další modely a projekty. Výsledky platí pro jeden model (Minimax) a jeden typ projektu (deterministická business logika). Replikace s jinými modely, nedeterministickými doménami (UI, strojové učení) a většími projekty by ukázala, které poznatky jsou přenositelné a které specifické pro tento případ.

Závěr

[DRAFT]

Tato práce se zabývala návrhem sady metrik a iterativního postupu pro systematické navrhování a vyhodnocování instrukcí pro AI coding agenty. Oba artefakty byly evaluovány na případové studii systému upomínek faktur, která zahrnovala pět pilotních iterací a čtyři ablační běhy.

Prvním cílem bylo navrhnout sadu metrik, která měří proces a kvalitu práce agenta, ne jen výsledek. Výsledná sada 19 metrik (procesní P1–P8, produktové Q1–Q8, efektivita E1–E3) pokrývá tři dimenze odvozené z taxonomie Fentona a Biemana. Metriky zachytily jak zlepšení při iterativních úpravách instrukcí, tak regrese způsobené nedeterminismem modelu i cílené dopady ablací. Kombinace deterministických a kvalitativních metrik (LLM-as-judge) přináší komplementární pohled — automatizované metriky nezachytí slabiny designu, kvalitativní samy o sobě nezměří procesní compliance.

Druhým cílem bylo iterativním postupem navrhnout instrukce, které dovedou agenta k dodržování stanovených exit kritérií. Cyklus Spuštění/Měření/Diagnóza/Úprava zlepšil procesní compliance ze 4/10 na 9/10 splněných deterministických kritérií ve dvou iteracích. Analýza průběhu odhalila opakující se vzorec: obecné pravidlo, které agent ignoroval, bylo nahrazeno konkrétním příkazem a následně doplněno verifikačním krokem. Každá iterace trvala 25–40 minut, postup je tedy proveditelný pro praktika bez specializovaného vybavení.

Třetím cílem bylo ablacemi identifikovat, které složky instrukční sady přispívají k měřenému chování agenta. Verifikační kroky se ukázaly jako neredundantní — jejich odebrání zhoršilo čtyři metriky a vedlo k výrazně vyššímu počtu testovacích cyklů. Sekce Package Quality byla pro automatizované metriky převážně redundantní, ale ovlivnila designovou kvalitu hodnocenou LLM-as-judge. Dominantním faktorem variability procesní compliance byl nedeterminismus modelu, ne obsah instrukcí.

Hlavním přínosem práce jsou přenositelné artefakty: sada metrik a iterativní postup, které může kdokoli aplikovat na vlastním projektu s vlastními prahy. Konkrétní instrukce a naměřené hodnoty platí pro tuto případovou studii. Práce je omezena na jeden model a jeden projekt — závěry mají povahu analytické generalizace na teorii, nikoliv statistické generalizace na populaci.

Bibliografie

- Agarwal, A., et al. (2024). PromptWizard: Task-Aware Prompt Optimization Framework [Microsoft Research. arXiv:2405.18369. Iterativní Score→Critique→Synthesize cyklus pro optimalizaci promptů.].
- Bockeler, B. (2025). *Understanding Spec-Driven-Development: Kiro, spec-kit, and TESSL* [Critical analysis of SDD tools. Spec-kit files “repetitive, both with each other, and with the code”]. <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>
- Breunig, D. (2025). Don’t Fight the Weights [Defines fighting-the-weights: when prompt instructions oppose trained model behavior; recognition signs and mitigation strategies]. <https://www.dbreunig.com/2025/11/11/don-t-fight-the-weights.html>
- Fenton, N. E., & Bieman, J. (2014). *Software Metrics: A Rigorous and Practical Approach* (3. vyd.) [Process vs. product vs. resource metrics taxonomy; GQM framework]. CRC Press.
- Gloaguen, T., Mündler, N., Müller, M., Raychev, V., & Vechev, M. (2025). Evaluating AGENTS.md: Are Repository-Level Context Files Helpful for Coding Agents? [LLM-generated context files reduce success rate; developer-written marginally help (+4%); context files increase cost 20%+; agents follow instructions but no performance gain; AGENTBENCH benchmark (138 instances, 12 repos)]. *arXiv preprint arXiv:2602.11988*. <https://arxiv.org/abs/2602.11988>
- Hassan, A. E., Li, H., Lin, D., Adams, B., Chen, T.-H., Kashiwa, Y., & Qiu, D. (2025). Agentic Software Engineering: Foundational Pillars and a Research Roadmap [SASE framework, BriefingScript, MentorScript, SE4H/SE4A duality]. *arXiv preprint arXiv:2509.06216*. <https://arxiv.org/abs/2509.06216>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design Science in Information Systems Research [Foundational DSR framework: build-evaluate cycle, 7 guidelines for design science research]. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can Language Models Resolve Real-World GitHub Issues? [2294 tasks from real GitHub repositories, de facto benchmark for AI coding agents]. *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2310.06770>
- Li, X., Chen, W., Liu, Y., Zheng, S., Chen, X., He, Y., Li, Y., et al. (2025). SkillsBench: Benchmarking How Well Agent Skills Work Across Diverse Tasks [Curated Skills +16.2pp; self-generated Skills -1.3pp; 2–3 focused modules optimal; comprehensive documentation hurts (-2.9pp); smaller model + Skills can exceed larger model without; 84 tasks, 11 domains, 7308 trajectories]. *arXiv preprint arXiv:2602.12670*. <https://arxiv.org/abs/2602.12670>
- Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the Middle: How Language Models Use Long Contexts [U-shaped performance curve; info in middle performs worse than no-context baseline]. *Transactions of the Association for Computational Linguistics*, 12.

- Lulla, K., Zhu, M., Kalliamvakou, E., & Mohylevskyy, Y. (2026). On the Impact of AGENTS.md Files on AI Coding Agents [124 PRs across 10 repos: -28% runtime, -20% output tokens with AGENTS.md; architectural info and coding conventions most effective]. *arXiv preprint arXiv:2601.20404*. <https://arxiv.org/abs/2601.20404>
- Mao, Y., He, J., & Chen, C. (2025). From Prompts to Templates: A Systematic Prompt Template Analysis for Real-world LLM Applications [Merged 4 frameworks (Google Cloud, Elavis Saravia, CRISPE, LangGPT); 7 component types; Directive 87% prevalence]. *Proceedings of the ACM International Conference on the Foundations of Software Engineering (FSE)*. <https://arxiv.org/abs/2504.02052>
- Mathews, N. S., & Nagappan, M. (2024). LLM-Based Test Generation as Bug Validation: Insights from Reproducing Real-World Bugs [68.1% generated test suites validate bugs instead of detecting them]. *Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware)*. <https://doi.org/10.1145/3664646.3664766>
- McCabe, T. J. (1976). A Complexity Measure. *IEEE Transactions on Software Engineering, SE-2*(4), 308–320. <https://doi.org/10.1109/TSE.1976.233837>
- METR. (2025). Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity [RCT: 16 developers, 246 tasks; AI tools slowed experienced developers by 19%; developers believed they were 24% faster]. *arXiv preprint arXiv:2507.09089*. <https://arxiv.org/abs/2507.09089>
- Panickssery, A., Bowman, S. R., & Feng, S. (2024). LLM Evaluators Recognize and Favor Their Own Generations [Causal link: self-recognition → self-preference; linear correlation; fine-tuning pushes recognition to 90%+]. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2404.13076>
- Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., & Harman, M. (2019). Mutation Testing Advances: An Analysis and Survey [Mutation score stronger predictor of fault detection than structural coverage]. *Advances in Computers, 112*, 275–378. <https://doi.org/10.1016/bs.adcom.2018.03.015>
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2008). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems, 24*(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>
- Prompt Alchemy: Automatic Prompt Refinement for Enhancing Code Generation [arXiv:2503.11085]. Iterativní refinement promptů pro code generation, evaluace přes execution.]. (2025).
- Razavi, S. P., & Fard, F. H. (2025). What Did I Do Wrong? Quantifying LLMs' Sensitivity and Consistency to Prompt Engineering [Up to 76 accuracy points difference from subtle formatting changes in few-shot settings]. *Proceedings of NAACL*. <https://arxiv.org/abs/2406.12334>
- Runeson, P., & Höst, M. (2009). Guidelines for Conducting and Reporting Case Study Research in Software Engineering. *Empirical Software Engineering, 14*(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- Shin, J., Tang, C., Mohati, T., Nayebi, M., Wang, S., & Hemmati, H. (2025). Prompt Engineering or Fine-Tuning: An Empirical Assessment of Large Language Models for Code. *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR)*. <https://arxiv.org/abs/2310.10508>

- Verga, P., Hofstätter, S., Cer, D., & Thorne, J. (2024). Replacing Judges with Juries: Evaluating LLM Generations with a Panel of Diverse Models [Panel of LLM evaluators (PoLL) outperforms single GPT-4 judge; disjoint model families reduce intra-model bias; 7x cheaper]. *arXiv preprint arXiv:2404.18796*. <https://arxiv.org/abs/2404.18796>
- Yin, R. K. (2018). *Case Study Research and Applications: Design and Methods* (6. vyd.) [Embedded single-case design; analytic generalization (to theory, not population)]. SAGE Publications.
- Yin, Z., Wang, J., Li, J., Shi, D., Wei, Y., Yue, Y., Liu, Z., Xia, X., & Lo, D. (2025). A Comprehensive Empirical Evaluation of Agent Frameworks on Code-centric Software Engineering Tasks. *arXiv preprint arXiv:2511.00872*.
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena [Foundational LLM-as-judge paper; position bias, verbosity bias, self-enhancement bias; GPT-4 >80% human agreement]. *Advances in Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2306.05685>

Přílohy

A. Formulář v plném znění

B. Zdrojové kódy výpočetních procedur

C. Baseline AGENTS.md

Baseline verze AGENTS.md (pilot-r1) a její evoluce do pilot-r3 jsou uvedeny v kapitole ?? (obrázky 4.1, ?? a 4.6).